

Curso Profesional de Entity Framework Core 8

Módulo 1 — Fundamentos de Entity Framework Core

TEORÍA

AUTOR: JAIME GALLO

.NET 8 · Entity Framework Core 8 · Visual Studio Community · SQL Server LocalDB · Proyecto AceriaData

Índice

Punto 1.1 – Qué es un ORM y por qué existe Entity Framework Core	3
Punto 1.2 – Arquitectura general de Entity Framework Core	6
Punto 1.3 – Componentes principales: DbContext, DbSet, Change Tracker, proveedores y migraciones	10
Punto 1.4 – El DbContext: rol, responsabilidades y propiedades DbSet	13
Punto 1.5 – Ciclo de vida del DbContext en aplicaciones de consola, web y servicios	16
Punto 1.6 – DbSet y operaciones básicas de acceso a datos	19
Punto 1.7 – Change Tracker: estados de las entidades y detección de cambios	26
Punto 1.8 – Gestión de entidades: Add, Update, Remove, Attach y Entry	31
Punto 1.9 – SaveChanges y unidad de trabajo	36
Punto 1.10 – Configuración inicial: opciones, cadena de conexión y logging	41
Punto 1.11 – Proveedores de datos: SQLite, SQL Server y PostgreSQL	46
Punto 1.12 – Integración de EF Core en aplicaciones .NET: inyección de dependencias y AddDbContext	49

Módulo 1 - Fundamentos de Entity Framework Core

Punto 1.1 - Qué es un ORM y por qué existe Entity Framework Core

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core.

Proyecto: Inicio del proyecto AceriaData. Se crea la solución, el proyecto de consola y se instalan los primeros paquetes de EF Core con SQL Server.

Objetivos de aprendizaje

Comprender qué es un ORM y qué problema resuelve.

Entender por qué Entity Framework Core es el ORM de Microsoft para .NET.

Identificar las diferencias entre acceso a datos manual con ADO.NET y acceso a datos con EF Core.

Crear la estructura inicial del proyecto AceriaData.

Instalar los paquetes necesarios de Entity Framework Core con SQL Server.

Ejecutar la primera verificación de que EF Core está correctamente referenciado.

Teoría

La persistencia y el modelo relacional

Toda aplicación que gestiona información necesita persistir esos datos en algún lugar. En una acería, las órdenes de fabricación, las planchas de acero y las aleaciones deben conservarse más allá del tiempo de ejecución del programa. La persistencia es el mecanismo que permite que los datos sobrevivan al cierre de la aplicación. Sin persistencia, cada vez que el programa se detuviera, toda la información se perdería.

Las bases de datos relacionales organizan la información en tablas compuestas por filas y columnas. Cada tabla representa una entidad del dominio, cada fila representa una instancia concreta y cada columna representa un atributo. Las relaciones entre tablas se establecen mediante claves foráneas. Un ejemplo típico sería una tabla OrdenesFabricacion con columnas Id, NumeroOrden, Cliente y FechaCreacion, y una tabla PlanchasAcero con una columna OrdenId que apunta a la orden a la que pertenece cada plancha.

```
CREATE TABLE OrdenesFabricacion (  
    Id INT PRIMARY KEY IDENTITY(1,1),  
    NumeroOrden NVARCHAR(50) NOT NULL,  
    Cliente NVARCHAR(200) NOT NULL,  
    FechaCreacion DATETIME2 NOT NULL  
);  
  
CREATE TABLE PlanchasAcero (  
    Id INT PRIMARY KEY IDENTITY(1,1),  
    OrdenId INT NOT NULL,  
    Espesor DECIMAL(18,2) NOT NULL,  
    Ancho DECIMAL(18,2) NOT NULL,  
    Largo DECIMAL(18,2) NOT NULL,  
    FOREIGN KEY (OrdenId) REFERENCES OrdenesFabricacion(Id)  
);
```

SQL

Este modelo lleva décadas siendo el estándar en aplicaciones empresariales por su robustez, su capacidad de consulta y su madurez. SQL es el lenguaje estándar para interactuar con estas bases de datos.

La brecha de impedancia

Los programas orientados a objetos organizan la información de forma distinta. En C#, una orden de fabricación se representa como una clase con propiedades:

CSHARP

```
public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; }
    public string Cliente { get; set; }
    public DateTime FechaCreacion { get; set; }
    public List<PlanchaAcero> Planchas { get; set; }
}

public class PlanchaAcero
{
    public int Id { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
}
```

El modelo de objetos usa clases, propiedades y referencias. El modelo relacional usa tablas, columnas y claves foráneas. Estos dos modelos no coinciden de forma natural. Esta diferencia se conoce como brecha de impedancia objeto-relacional. Traducir entre ambos mundos manualmente implica escribir código repetitivo: abrir conexiones, construir sentencias SQL, mapear columnas a propiedades, gestionar transacciones y cerrar recursos.

Un ejemplo de esa traducción manual con ADO.NET sería el siguiente:

CSHARP

```
using var connection = new SqlConnection("Server=(localdb)\
\\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;");
connection.Open();

var command = connection.CreateCommand();
command.CommandText = "SELECT Id, NumeroOrden, Cliente, FechaCreacion FROM OrdenesFabricacion";

using var reader = command.ExecuteReader();
var ordenes = new List<OrdenFabricacion>();

while (reader.Read())
{
    ordenes.Add(new OrdenFabricacion
    {
        Id = reader.GetInt32(0),
        NumeroOrden = reader.GetString(1),
        Cliente = reader.GetString(2),
        FechaCreacion = reader.GetDateTime(3)
    });
}
```

Este código funciona, pero es repetitivo y propenso a errores. Cada consulta requiere construir el SQL, ejecutarlo, leer los resultados y mapearlos manualmente. Este código no aporta valor al negocio.

Qué es un ORM

Un ORM (Object-Relational Mapper) es una herramienta que automatiza la traducción entre el modelo de objetos de la aplicación y el modelo relacional de la base de datos. Permite trabajar con la base de datos utilizando clases y objetos de C#, sin escribir SQL manualmente para las operaciones habituales. El ORM se encarga de generar las sentencias SQL, ejecutarlas, leer los resultados y materializarlos de vuelta en objetos.

Con EF Core, la consulta anterior se escribe así:

CSHARP

```
using var context = new AceriaDbContext();
var ordenes = context.OrdenesFabricacion.ToList();
```

EF Core genera el SELECT, abre la conexión, ejecuta la consulta, lee los resultados y construye la lista de objetos OrdenFabricacion. El programador no escribe SQL ni gestiona la conexión.

Un ORM cubre las operaciones fundamentales de persistencia: insertar, consultar, actualizar y eliminar registros. También gestiona las relaciones entre entidades, las transacciones, el seguimiento de cambios y la generación del esquema. En las consultas, traduce expresiones escritas en el lenguaje de programación a SQL. En las escrituras, detecta qué propiedades han cambiado y genera las sentencias de actualización correspondientes.

Ventajas de usar un ORM

El uso de un ORM aporta varias ventajas. Reduce la cantidad de código repetitivo. Disminuye la superficie de errores al no construir SQL manualmente. Proporciona una capa de abstracción que reduce parte del código específico del motor, aunque cambiar de proveedor puede exigir revisar tipos, funciones, consultas y migraciones. También facilita distintas estrategias de testing, pero un proveedor de prueba no reproduce necesariamente el comportamiento de SQL Server. Además, ofrece herramientas de migración que versionan el esquema.

Estas ventajas no eliminan la necesidad de conocer SQL. Un programador que entiende cómo funcionan las consultas SQL puede diagnosticar problemas de rendimiento y escribir consultas más eficientes. EF Core genera SQL, y conocer ese SQL es parte del trabajo profesional.

Entity Framework Core como ORM de .NET

Entity Framework Core es un ORM ligero, extensible y multiplataforma para .NET. Es la versión moderna de Entity Framework, rediseñada desde cero para .NET Core y evolucionada hasta .NET 8. Funciona en Windows, Linux y macOS. Soporta múltiples motores de base de datos mediante proveedores. Se distribuye como paquetes NuGet y se integra con el contenedor de inyección de dependencias de .NET.

EF Core ofrece varias características que lo definen. Permite trabajar con entidades de C# sin modificar su estructura. Traduce consultas LINQ a SQL. Gestiona el seguimiento de cambios de las entidades. Proporciona migraciones para versionar el esquema. Soporta relaciones uno a uno, uno a muchos y muchos a muchos. Permite configurar el modelo mediante convenciones, Data Annotations o Fluent API. Y ofrece herramientas de diagnóstico para analizar el SQL generado.

Diferencias con ADO.NET

ADO.NET es la tecnología de acceso a datos de bajo nivel de .NET. Proporciona clases como `SqlConnection`, `SqlCommand` y `SqlDataReader` para ejecutar SQL manualmente. ADO.NET ofrece control total sobre el SQL, pero exige escribir todo el código de conexión, mapeo y gestión de recursos.

EF Core se construye sobre ADO.NET y automatiza esas tareas. La elección entre ambos depende del escenario: ADO.NET para consultas muy específicas y optimizadas, EF Core para la mayor parte del acceso a datos en aplicaciones empresariales. EF Core no sustituye a ADO.NET, lo encapsula.

SQL Server Express LocalDB

SQL Server Express LocalDB es una versión ligera del motor de SQL Server que puede instalarse como componente del entorno de Visual Studio Community. Se ejecuta en modo usuario, sin necesidad de configuración de servidor ni de servicios en segundo plano. LocalDB crea los archivos de base de datos .mdf en el directorio del usuario y se inicia bajo demanda cuando una cadena de conexión lo requiere.

La cadena de conexión típica para LocalDB tiene el siguiente formato:

```
Server=(localdb)\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;
```

TEXT

`Server=(localdb)\mssqllocaldb` → indica la instancia de LocalDB por defecto.

`Database=AceriaDB` → nombre de la base de datos que se creará.

`Trusted_Connection=True` → usa la autenticación de Windows del usuario actual.

LocalDB es ideal para desarrollo porque no requiere instalar un servidor completo y se integra directamente con Visual Studio a través del Explorador de objetos de SQL Server.

Versiones de EF Core

Entity Framework Core ha evolucionado desde su primera versión en 2016. Cada versión ha añadido características y mejorado el rendimiento. EF Core 8 es la versión que se usa en este curso, alineada con .NET 8. Es una versión LTS, con soporte a largo plazo. Incluye mejoras en la traducción de consultas, en el rendimiento de las migraciones y en las herramientas de diagnóstico.

La versión se elige al instalar el paquete NuGet correspondiente. En el archivo .csproj del proyecto se especifica:

```
<PackageReference Include="Microsoft.EntityFrameworkCore.SqlServer" Version="8.0.31" />
```

XML

El paquete `Microsoft.EntityFrameworkCore.SqlServer` es el proveedor de EF Core para SQL Server y Azure SQL. Se invoca con el método `UseSqlServer` al configurar el `DbContext`.

Si se instala una versión distinta a la 8.x, pueden aparecer conflictos con el framework destino o con otros paquetes.

Cuándo usar EF Core

EF Core es adecuado en la mayoría de aplicaciones empresariales. Es especialmente útil cuando el modelo de datos es complejo, cuando se trabaja con muchas entidades relacionadas o cuando se necesita portabilidad entre motores de base de datos. También es adecuado cuando se quiere reducir el código de acceso a datos y centrar el esfuerzo en la lógica de negocio.

En escenarios de altísimo rendimiento con consultas muy específicas, puede combinarse con SQL manual o con Dapper. EF Core es la opción por defecto en aplicaciones .NET modernas.

El proyecto del curso

A lo largo del curso se construye una aplicación de persistencia para la gestión de órdenes de fabricación de planchas de acero en una acería. El proyecto se llama AceriaData. Comienza como una aplicación de consola y evoluciona hasta convertirse en una capa de persistencia completa. Cada punto del temario añade una pieza al proyecto. Al final del curso, el alumno tiene una aplicación funcional con entidades, relaciones, consultas, optimizaciones, transacciones, migraciones y tests.

El dominio del proyecto incluye varias entidades. La orden de fabricación representa una solicitud de producción de planchas. La plancha de acero representa una unidad fabricada. La aleación representa la composición química del acero. El estado de la orden representa la situación en la que se encuentra. Estas entidades se irán definiendo y relacionando a lo largo de los módulos. En este primer punto solo se prepara la estructura del proyecto.

Resumen de la teoría

La persistencia permite que los datos sobrevivan al cierre de la aplicación.

El modelo relacional y el modelo de objetos no coinciden: brecha de impedancia.

Un ORM automatiza la traducción entre ambos modelos.

Entity Framework Core es el ORM de Microsoft para .NET.

SQL Server Express LocalDB se instala con Visual Studio y es ideal para desarrollo.

El proyecto del curso se llama AceriaData y es creciente.

Punto 1.2 - Arquitectura general de Entity Framework Core

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core.

Proyecto: Se añade al proyecto AceriaData el primer DbContext y la primera entidad, conectados a SQL Server LocalDB.

Objetivos de aprendizaje

Identificar los componentes que forman la arquitectura de Entity Framework Core.

Comprender cómo se relacionan entre sí el DbContext, las entidades, el Change Tracker, los proveedores y las migraciones.

Entender el flujo que sigue una operación desde que se escribe en C# hasta que se ejecuta en la base de datos.

Crear el primer DbContext en el proyecto AceriaData.

Configurar la cadena de conexión a SQL Server LocalDB.

Ejecutar la primera creación de base de datos con EnsureCreated.

Teoría

La arquitectura como capas de responsabilidad

Entity Framework Core no es un bloque monolítico. Es un conjunto de componentes que colaboran entre sí, cada uno con una responsabilidad concreta. Comprender esa separación de responsabilidades permite diagnosticar errores con precisión y elegir la configuración adecuada en cada escenario.

En la parte superior de la arquitectura se encuentra el código de la aplicación: las entidades, los servicios y el DbContext. Ese código no habla directamente con SQL Server. Habla con la API pública de EF Core. Por debajo de esa API hay un conjunto de servicios internos que se encargan de traducir las operaciones a SQL, gestionar el estado de las entidades y comunicarse con el proveedor de base de datos.

```
public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
}

public class AceriaDbContext : DbContext
{
    public DbSet<OrdenFabricacion> OrdenesFabricacion { get; set; } = null!;

    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
        optionsBuilder.UseSqlServer(
            "Server=(localdb)\\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;");
    }
}
```

CSHARP

La clase AceriaDbContext representa la puerta de entrada a toda la infraestructura de EF Core. Desde ella se accede al Change Tracker, al modelo, al proveedor y a la conexión.

El DbContext como punto de entrada

El DbContext es la clase central de la arquitectura. Cada instancia de DbContext representa una sesión con la base de datos. Esa sesión mantiene una conexión lógica, un Change Tracker, una caché de entidades cargadas y una configuración del modelo.

El DbContext expone colecciones de entidades a través de propiedades DbSet<T>. Cada DbSet representa una tabla y permite consultar, insertar, actualizar y eliminar entidades de ese tipo. La expresión context.OrdenesFabricacion no ejecuta ninguna consulta por sí misma. Solo prepara una consulta que se ejecutará cuando se itere sobre ella o se materialice.

```
using var context = new AceriaDbContext();
var ordenes = context.OrdenesFabricacion.Where(o => o.Cliente == "Constructora del Norte").ToList();
```

CSHARP

La primera línea crea el DbContext. La segunda construye una consulta y la materializa con ToList. Solo en ese momento se ejecuta el SQL contra SQL Server.

El modelo de entidades

El modelo es la representación en memoria del esquema de la base de datos. Está formado por las entidades, sus propiedades, sus relaciones, sus claves y sus restricciones. EF Core construye el modelo la primera vez que se crea una instancia del DbContext, a partir de las convenciones, las Data Annotations y la configuración de Fluent API.

El modelo se almacena en una estructura interna que EF Core usa para traducir operaciones. Cuando se escribe context.OrdenesFabricacion.Where(o => o.Cliente == "Constructora del Norte"), EF Core consulta el modelo para saber que OrdenFabricacion se mapea a la tabla OrdenesFabricacion, que Cliente se mapea a la columna Cliente y que el filtro se traduce a una cláusula WHERE.

```
public class PlanchaAcero
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public OrdenFabricacion Orden { get; set; } = null!;
}
```

CSHARP

La propiedad Orden es una propiedad de navegación. EF Core la interpreta como una relación con OrdenFabricacion y genera la clave foránea correspondiente en la tabla PlanchasAcero.

El Change Tracker

El Change Tracker es el componente que realiza un seguimiento de las entidades cargadas desde la base de datos. Cada entidad tiene un estado asociado: Added, Unchanged, Modified, Deleted o Detached. Cuando una propiedad cambia, el Change Tracker lo detecta y actualiza el estado de la entidad.

El Change Tracker proporciona a SaveChanges el estado necesario para decidir qué operaciones de persistencia deben ejecutarse. Una entidad Added provoca una inserción, una Modified una actualización y una Deleted una eliminación; el SQL concreto lo genera el proveedor relacional configurado.

```
var orden = new OrdenFabricacion { NumeroOrden = "OF-001", Cliente = "Constructora del Norte" };
context.OrdenesFabricacion.Add(orden);
context.SaveChanges();
```

CSHARP

La llamada a Add registra la entidad en el Change Tracker con estado Added. La llamada a SaveChanges inspecciona el Change Tracker, genera el INSERT y lo ejecuta contra SQL Server.

Los proveedores de base de datos

El proveedor es el componente que traduce las operaciones abstractas de EF Core a SQL específico de un motor. Existen proveedores oficiales para SQL Server, SQLite, PostgreSQL, MySQL y Azure Cosmos DB, entre otros. Cada proveedor conoce las particularidades del motor al que se conecta: tipos de datos, funciones, sintaxis de paginación y comportamiento de transacciones.

El proveedor se configura al construir el DbContext, mediante el método de extensión correspondiente:

```
optionsBuilder.UseSqlServer("Server=(localdb)\\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;");
```

CSHARP

UseSqlServer es el método que registra el proveedor de SQL Server. Si se usara SQLite, se llamaría a UseSqlite. El resto del código de la aplicación no cambia.

El proveedor de SQL Server

El proveedor de SQL Server se distribuye en el paquete Microsoft.EntityFrameworkCore.SqlServer. Internamente utiliza Microsoft.Data.SqlClient para abrir conexiones y ejecutar comandos. Es compatible con SQL Server 2012 y versiones posteriores, con Azure SQL Database y con SQL Server Express LocalDB.

La cadena de conexión tiene el siguiente formato:

```
Server=(localdb)\\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;
```

TEXT

Server → indica la instancia de SQL Server. (localdb)\\mssqllocaldb es la instancia por defecto de LocalDB.

Database → nombre de la base de datos. Si no existe, EF Core la crea cuando se llama a EnsureCreated o se aplican migraciones.

Trusted_Connection=True → usa la autenticación de Windows del usuario actual.

Las migraciones

Las migraciones son el mecanismo que permite versionar el esquema de la base de datos. Cada migración representa un conjunto de cambios en el modelo que se traducen a operaciones SQL para crear o modificar tablas, columnas, índices y restricciones. Las migraciones se generan a partir del modelo de entidades y se aplican de forma incremental.

Una migración se compone de dos archivos: uno con el código que aplica los cambios y otro con el código que los revierte. Ambos se generan automáticamente a partir del modelo.

```
migrationBuilder.CreateTable(
    name: "OrdenesFabricacion",
    columns: table => new
    {
        Id = table.Column<int>(nullable: false)
            .Annotation("SqlServer:Identity", "1, 1"),
```

CSHARP

```

NumeroOrden = table.Column<string>(nullable: false),
Cliente = table.Column<string>(nullable: false),
FechaCreacion = table.Column<DateTime>(nullable: false)
},
constraints: table =>
{
    table.PrimaryKey("PK_OrdenesFabricacion", x => x.Id);
});

```

Este código es generado por EF Core. No se escribe a mano. Se revisa para verificar que los cambios son los esperados.

Las consultas LINQ

EF Core permite escribir consultas utilizando LINQ. Estas consultas se escriben contra las propiedades DbSet y se traducen a SQL por el proveedor. LINQ permite filtrar, ordenar, proyectar, agrupar y unir datos de forma tipada y segura.

La consulta no se ejecuta hasta que se itera sobre ella o se materializa con métodos como ToList, FirstOrDefault o Count. Este comportamiento se conoce como ejecución diferida.

```

var consulta = context.OrdenesFabricacion.Where(o => o.Cliente == "Constructora del Norte");
var lista = consulta.ToList();

```

CSHARP

La primera línea construye el árbol de expresión. La segunda lo traduce a SQL, lo ejecuta y materializa los resultados.

El flujo completo de una operación

Cuando se ejecuta una operación en EF Core, se sigue un flujo concreto. Primero, el código de la aplicación construye una consulta o modifica una entidad. Segundo, el DbContext recibe esa operación y consulta el modelo para saber cómo traducirla. Tercero, el proveedor genera el SQL correspondiente. Cuarto, se abre la conexión, se ejecuta el comando y se leen los resultados. Quinto, los resultados se materializan como objetos de C# y se devuelven a la aplicación.

```

var orden = context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == "0F-001");

```

CSHARP

Esta línea activa todo el flujo: construye la consulta, consulta el modelo, genera el SQL, abre la conexión, ejecuta el SELECT TOP 1, lee la fila, materializa el objeto y lo devuelve. Si no hay coincidencia, devuelve null.

La API pública y los servicios internos

La API pública de EF Core está formada por las clases DbContext, DbSet<T>, DbContextOptions y los métodos de extensión de LINQ. Es lo que el programador usa directamente.

Por debajo, EF Core tiene un conjunto de servicios internos que se encargan del trabajo pesado: el servicio de modelo, el servicio de cambio, el servicio de consulta, el servicio de actualización, el servicio de conexión y el servicio de migraciones. Estos servicios se resuelven mediante inyección de dependencias interna.

Integración con .NET

EF Core se integra con .NET a través del contenedor de inyección de dependencias. El DbContext se registra con AddDbContext, especificando el proveedor y las opciones de conexión. También se puede configurar el logging, el comportamiento de seguimiento de cambios y otras opciones.

```

services.AddDbContext<AceriaDbContext>(options =>
    options.UseSqlServer("Server=(localdb)\\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;"));

```

CSHARP

Esta línea registra el DbContext en el contenedor con ámbito por petición. Cada petición recibe una instancia nueva del DbContext.

El proyecto AceriaData

En el proyecto AceriaData, en este punto la arquitectura de EF Core se materializa con la primera entidad del dominio, OrdenFabricacion, el DbContext AceriaDbContext y la configuración del proveedor de SQL Server. PlanchaAcero y Aleacion se incorporarán en 1.3, y EstadoOrden en 1.4, manteniendo así la evolución acumulativa del proyecto. A lo largo de los siguientes puntos se profundizará en el Change Tracker, las migraciones, las consultas y las optimizaciones.

Resumen de la teoría

EF Core se compone de DbContext, modelo, Change Tracker, proveedores, migraciones y consultas.

El DbContext es el punto de entrada y representa una sesión con la base de datos.

El modelo es la representación en memoria del esquema.

El Change Tracker detecta cambios y genera las sentencias SQL correspondientes.

El proveedor traduce las operaciones abstractas a SQL específico del motor.

Las migraciones versionan el esquema.

Las consultas LINQ se traducen a SQL y se ejecutan de forma diferida.

El DbContext se integra con .NET mediante inyección de dependencias.

Punto 1.3 - Componentes principales: DbContext, DbSet, Change Tracker, proveedores y migraciones

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core.

Proyecto: Se identifican y utilizan de forma explícita cada uno de los componentes de EF Core dentro del proyecto AceriaData, conectado a SQL Server LocalDB.

Objetivos de aprendizaje

Reconocer el rol de cada componente de EF Core dentro del código del proyecto.

Comprender la diferencia entre DbContext y DbSet.

Identificar el Change Tracker y sus estados.

Entender el papel del proveedor de SQL Server en la cadena de conexión.

Introducir las migraciones como mecanismo de versionado del esquema.

Modificar el proyecto AceriaData para observar el comportamiento de cada componente.

Teoría

Los cinco componentes y su responsabilidad

Entity Framework Core se apoya en cinco componentes que colaboran entre sí. El DbContext es el coordinador de la sesión con la base de datos. El DbSet es la puerta de acceso a cada tabla. El Change Tracker es el supervisor que registra los cambios. El proveedor es el traductor al SQL específico del motor. Las migraciones son el mecanismo que versiona el esquema.

Cada uno tiene una responsabilidad clara y no se solapa con las demás. El DbContext no traduce a SQL: eso lo hace el proveedor. El DbSet no detecta cambios: eso lo hace el Change Tracker. Las migraciones no ejecutan consultas: eso lo hace el DbSet a través del DbContext.

```
public class AceriaDbContext : DbContext
{
    public DbSet<OrdenFabricacion> OrdenesFabricacion { get; set; } = null!;
    public DbSet<PlanchaAcero> PlanchasAcero { get; set; } = null!;

    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
        optionsBuilder.UseSqlServer(
```

CSHARP

```

    }
    "Server=(localdb)\\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;");
}

```

En este fragmento aparecen tres de los cinco componentes: el DbContext (AceriaDbContext), los DbSet (OrdenesFabricacion y PlanchasAcero) y el proveedor (invocado por UseSqlServer). Los otros dos, el Change Tracker y las migraciones, se activan cuando se ejecutan operaciones.

El DbContext

El DbContext es la clase que representa una sesión con la base de datos. Agrupa los DbSet, mantiene el Change Tracker, conserva la configuración del modelo y gestiona la conexión. Cada instancia de DbContext tiene un ciclo de vida corto: se crea, se usa y se libera.

El DbContext también expone propiedades y métodos útiles como Database, ChangeTracker, Model y SaveChanges. La propiedad Database permite crear, eliminar o consultar la base de datos. La propiedad ChangeTracker da acceso al supervisor de cambios. La propiedad Model expone el modelo construido.

```

using var context = new AceriaDbContext();
Console.WriteLine($"Proveedor: {context.Database.ProviderName}");
Console.WriteLine($"Entidades en el modelo: {context.Model.GetEntityTypes().Count()}");

```

CSHARP

La primera línea crea el DbContext. La segunda muestra el nombre del proveedor configurado. La tercera recorre el modelo y cuenta las entidades registradas.

El DbSet

El DbSet es la representación de una tabla dentro del DbContext. Cada propiedad DbSet<T> expone una colección de entidades del tipo T. Sobre un DbSet se pueden ejecutar operaciones de consulta con LINQ, y operaciones de escritura con Add, Update y Remove.

Un DbSet no contiene datos en memoria. Es una abstracción que permite construir consultas y registrar operaciones. Los datos se cargan cuando se materializa la consulta.

```

var consulta = context.OrdenesFabricacion.Where(o => o.Cliente == "Constructora del Norte");
var lista = consulta.ToList();

```

CSHARP

La primera línea construye la consulta. La segunda la ejecuta y materializa los resultados. Antes de ToList, no se ha ejecutado ningún SQL.

El Change Tracker

El Change Tracker es el componente que registra el estado de cada entidad que el DbContext conoce. Cada entidad tiene uno de cinco estados: Added, Unchanged, Modified, Deleted o Detached. El estado determina qué sentencia SQL se generará al llamar a SaveChanges.

Cuando una entidad se añade con Add, pasa al estado Added. Cuando se carga desde la base de datos, pasa al estado Unchanged. Cuando se modifica una propiedad, pasa al estado Modified. Cuando se elimina con Remove, pasa al estado Deleted. Cuando se desvincula del contexto, pasa al estado Detached.

```

var orden = new OrdenFabricacion { NumeroOrden = "OF-001", Cliente = "Constructora del Norte" };
Console.WriteLine(context.Entry(orden).State);
context.OrdenesFabricacion.Add(orden);
Console.WriteLine(context.Entry(orden).State);

```

CSHARP

La primera línea crea la entidad. La segunda muestra su estado inicial, que es Detached porque aún no está registrada. La tercera la añade al DbContext. La cuarta muestra el nuevo estado, que es Added.

El proveedor

El proveedor es el componente que traduce las operaciones abstractas de EF Core a SQL específico de un motor concreto. El proveedor de SQL Server se registra con UseSqlServer. El proveedor de SQLite se registra con UseSqlite. El proveedor de PostgreSQL se registra con UseNpgsql.

El proveedor también expone la propiedad ProviderName, que devuelve el nombre completo del ensamblado del proveedor. Es útil para verificar qué motor está configurado.

```
Console.WriteLine(context.Database.ProviderName);
```

CSHARP

Esta línea imprime Microsoft.EntityFrameworkCore.SqlServer, confirmando que el proveedor activo es el de SQL Server.

Las migraciones

Las migraciones son el mecanismo que versiona el esquema de la base de datos. Cada migración es una clase generada a partir del modelo, que contiene métodos Up y Down. El método Up aplica los cambios. El método Down los revierte.

Las migraciones se generan con el comando `dotnet ef migrations add`, se aplican con `dotnet ef database update` y se listan con `dotnet ef migrations list`. Estas herramientas requieren el paquete Microsoft.EntityFrameworkCore.Design y la herramienta global dotnet-ef.

```
dotnet ef migrations add InitialCreate
dotnet ef database update
```

BASH

El primer comando genera una migración llamada InitialCreate. El segundo la aplica a la base de datos.

DbContext frente a DbSet

El DbContext agrupa todos los DbSet y coordina la sesión. El DbSet representa una tabla concreta. El DbContext es el punto de entrada al Change Tracker, al modelo, a la conexión y a las migraciones. El DbSet solo permite construir consultas y registrar operaciones sobre un tipo de entidad.

Un DbContext puede tener muchos DbSet. Cada DbSet corresponde a una entidad del dominio. En el proyecto AceriaData, este punto consolida OrdenesFabricacion y añade PlanchasAcero y Aleaciones. EstadosOrden se incorporará en 1.4.

Change Tracker en acción

El Change Tracker detecta cambios comparando los valores actuales de las propiedades con los valores originales cargados desde la base de datos. Para entidades conectadas (tracked), esta detección se realiza al llamar a SaveChanges. Para entidades desconectadas (detached), es necesario indicar el estado manualmente.

La propiedad ChangeTracker.Entries() devuelve una colección con todas las entidades rastreadas. Esta colección permite inspeccionar el estado de cada entidad antes de guardar cambios.

```
var entradas = context.ChangeTracker.Entries();
foreach (var entrada in entradas)
{
    Console.WriteLine($"{entrada.Entity.GetType().Name}: {entrada.State}");
}
```

CSHARP

La primera línea obtiene todas las entidades rastreadas. El bucle imprime el nombre del tipo y su estado.

Proveedores disponibles

EF Core ofrece proveedores oficiales para los motores más usados. Microsoft.EntityFrameworkCore.SqlServer es el proveedor para SQL Server y Azure SQL. Microsoft.EntityFrameworkCore.Sqlite es el proveedor para SQLite. Npgsql.EntityFrameworkCore.PostgreSQL es el proveedor para PostgreSQL. Pomelo.EntityFrameworkCore.MySql es el proveedor para MySQL.

Cada proveedor tiene sus propias limitaciones y características. El proveedor de SQL Server soporta tipos como datetime2, decimal y nvarchar. El proveedor de SQLite no tiene un tipo decimal nativo y lo almacena como TEXT. Estas diferencias influyen en la portabilidad del código entre motores.

Migraciones y control de versiones

Los archivos de migración se incluyen en el control de versiones del proyecto. Cada migración representa un cambio concreto en el esquema. Aplicar las migraciones en orden reconstruye el esquema completo. Esto permite que varios desarrolladores trabajen sobre el mismo modelo sin conflictos, y que los entornos de desarrollo, pruebas y producción se mantengan sincronizados.

Los archivos generados se almacenan en la carpeta Migrations del proyecto. Cada migración incluye la fecha y hora de creación en el nombre, lo que garantiza el orden de aplicación.

TEXT

```
Migrations/
  20240115120000_InitialCreate.cs
  20240115120000_InitialCreate.Designer.cs
  AceriaDbContextModelSnapshot.cs
El archivo InitialCreate.cs contiene los métodos Up y Down. El archivo .Designer.cs contiene los metadatos de la migración. El archivo AceriaDbContextModelSnapshot.cs contiene el estado actual del modelo.

El modelo y su construcción
El modelo se construye la primera vez que se crea una instancia del DbContext. EF Core recorre las entidades, aplica las convenciones, lee las Data Annotations y ejecuta la configuración de Fluent API. El resultado es una estructura interna que se almacena en caché durante el resto del proceso.

El modelo se puede inspeccionar a través de la propiedad Model. Esta propiedad expone las entidades registradas, sus propiedades y sus relaciones. Es útil para diagnosticar problemas de configuración.

csharp
var entidad = context.Model.FindEntityType(typeof(OrdenFabricacion));
Console.WriteLine($"Tabla: {entidad!.GetTableName()}");
```

La primera línea busca la entidad OrdenFabricacion en el modelo. La segunda muestra el nombre de la tabla a la que se mapea.

El proyecto AceriaData

En el proyecto AceriaData, los cinco componentes de EF Core quedan ya representados de forma explícita. El DbContext es AceriaDbContext. En este punto el modelo incorpora OrdenesFabricacion, PlanchasAcero y Aleaciones. El Change Tracker participa al registrar entidades, el proveedor es Microsoft.EntityFrameworkCore.SqlServer y las primeras migraciones se generan y aplican sobre AceriaDB en LocalDB.

Resumen de la teoría

EF Core se compone de DbContext, DbSet, Change Tracker, proveedores y migraciones.

El DbContext coordina la sesión con la base de datos.

El DbSet representa una tabla concreta.

El Change Tracker registra el estado de las entidades.

El proveedor traduce las operaciones a SQL específico del motor.

Las migraciones versionan el esquema de la base de datos.

El modelo se construye la primera vez que se crea el DbContext.

En el proyecto AceriaData ya están representados los cinco componentes estudiados y el modelo alcanza el estado previsto para 1.3.

Punto 1.4 - El DbContext: rol, responsabilidades y propiedades DbSet

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core.

Proyecto: Se profundiza en el DbContext de AceriaData, se inspecciona Aleacion incorporada en 1.3 y se añade EstadoOrden con su DbSet y migración.

Objetivos de aprendizaje

Comprender el rol del DbContext como coordinador de la sesión con la base de datos.

Identificar las responsabilidades del DbContext: modelo, conexión, Change Tracker y SaveChanges.

Conocer las propiedades principales del DbContext: Database, Model, ChangeTracker y SaveChanges.

Entender la diferencia entre el DbContext y los DbSet que contiene.

Configurar el DbContext mediante DbContextOptions.

Comprender cómo el DbContext expone los DbSet existentes y añadir el DbSet de EstadoOrden al proyecto AceriaData.

Teoría

El DbContext como coordinador de la sesión

El DbContext es la clase que representa una sesión con la base de datos. Cada instancia de DbContext agrupa una conexión, un modelo, un Change Tracker y una configuración. Toda operación de EF Core pasa por el DbContext. Las consultas se construyen sobre sus DbSet. Las escrituras se registran en su Change Tracker. Las migraciones se aplican a través de su propiedad Database.

Un DbContext no es un repositorio. No encapsula una tabla concreta. Es el coordinador de todas las operaciones que la aplicación realiza contra la base de datos durante un periodo de tiempo determinado. Su ciclo de vida está pensado para ser corto: se crea, se usa y se libera.

```
using var context = new AceriaDbContext();

var ordenes = context.OrdenesFabricacion.ToList();
var planchas = context.PlanchasAcero.ToList();

context.SaveChanges();
```

CSHARP

En este fragmento, un único DbContext coordina dos consultas y una operación de guardado. Todas las entidades cargadas quedan registradas en el Change Tracker de esa instancia.

Responsabilidades del DbContext

El DbContext tiene cuatro responsabilidades principales. La primera es exponer las entidades a través de propiedades DbSet<T>. La segunda es mantener la configuración del modelo. La tercera es gestionar el Change Tracker. La cuarta es coordinar la conexión con la base de datos y las operaciones de guardado.

Cuando se crea una instancia del DbContext, EF Core construye el modelo la primera vez y lo almacena en caché. El modelo incluye las entidades, sus propiedades, sus claves, sus relaciones y sus restricciones. Todas las operaciones posteriores usan ese modelo para traducir consultas y actualizaciones.

La propiedad Database

La propiedad Database da acceso a las operaciones sobre la base de datos. Expone métodos como EnsureCreated, EnsureDeleted, Migrate y CanConnect. También expone propiedades como ProviderName, que devuelve el nombre del proveedor configurado.

```
Console.WriteLine(context.Database.ProviderName);
context.Database.EnsureCreated();
```

CSHARP

La primera línea muestra el nombre del proveedor. La segunda crea la base de datos y las tablas si no existen.

La propiedad Model

La propiedad Model expone el modelo construido por EF Core. Permite inspeccionar las entidades registradas, sus propiedades, sus claves y sus relaciones. Es útil para diagnosticar problemas de configuración y para verificar que el modelo es el esperado.

```
foreach (var entidad in context.Model.GetEntityTypes())
{
    Console.WriteLine($"{entidad.ClrType.Name} → {entidad.GetTableName()}");
}
```

CSHARP

El bucle recorre cada entidad del modelo y muestra el nombre de la clase y el nombre de la tabla a la que se mapea.

La propiedad ChangeTracker

La propiedad ChangeTracker da acceso al supervisor de cambios. Expone la colección Entries(), que devuelve todas las entidades rastreadas con su estado. También expone métodos como Clear(), que desvincula todas las entidades del contexto.

CSHARP

```
var entradas = context.ChangeTracker.Entries();
foreach (var entrada in entradas)
{
    Console.WriteLine($"{entrada.Entity.GetType().Name}: {entrada.State}");
}
```

La primera línea obtiene todas las entidades rastreadas. El bucle muestra el tipo y el estado de cada una.

La propiedad SaveChanges

El método SaveChanges es el punto en el que EF Core traduce los cambios del Change Tracker a sentencias SQL y los ejecuta. Devuelve el número de filas afectadas. Si ocurre un error, lanza una excepción y no se aplica ningún cambio.

CSHARP

```
var entradasEscritas = context.SaveChanges();
Console.WriteLine($"Entradas escritas: {entradasEscritas}");
```

La primera línea guarda los cambios y devuelve el número de filas afectadas. La segunda muestra ese número.

Los DbSet dentro del DbContext

Un DbSet es la representación de una tabla dentro del DbContext. Cada propiedad DbSet<T> expone una colección de entidades del tipo T. Un DbContext puede tener muchos DbSet, uno por cada entidad del dominio.

Un DbSet no es una lista. No contiene datos en memoria hasta que se materializa una consulta. Las operaciones sobre un DbSet se traducen a SQL cuando se itera o se materializa el resultado.

CSHARP

```
public DbSet<OrdenFabricacion> OrdenesFabricacion { get; set; } = null!;
public DbSet<PlanchaAcero> PlanchasAcero { get; set; } = null!;
public DbSet<Aleacion> Aleaciones { get; set; } = null!;
```

Cada propiedad expone una tabla. El DbContext las agrupa y las pone a disposición del resto de la aplicación.

DbContextOptions

El DbContext se configura mediante un objeto DbContextOptions. Este objeto contiene el proveedor, la cadena de conexión, el logging, el comportamiento de seguimiento y otras opciones. Las opciones se pasan al constructor del DbContext o se configuran en el método OnConfiguring.

CSHARP

```
public class AceriaDbContext : DbContext
{
    public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }

    public DbSet<OrdenFabricacion> OrdenesFabricacion { get; set; } = null!;
}
```

En este ejemplo, el DbContext recibe las opciones por constructor. Este patrón es el recomendado en aplicaciones que usan inyección de dependencias.

Configuración en OnConfiguring

Cuando el DbContext no recibe opciones por constructor, se puede configurar en el método OnConfiguring. Este método se ejecuta una vez por cada instancia del DbContext.

CSHARP

```
protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
{
    if (!optionsBuilder.IsConfigured)
    {
        optionsBuilder.UseSqlServer(
            "Server=(localdb)\\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;");
    }
}
```

La condición IsConfigured evita sobrescribir la configuración si ya se ha pasado por constructor. Es una salvaguarda útil en aplicaciones que combinan ambos enfoques.

El ciclo de vida del DbContext

El DbContext está diseñado para vivir poco tiempo. En aplicaciones de consola, se crea una instancia por operación. En aplicaciones web, se registra con ámbito por petición. En servicios de larga duración, se crea una instancia por unidad de trabajo.

Mantener un DbContext vivo durante mucho tiempo provoca que el Change Tracker acumule entidades, consuma memoria y devuelva datos obsoletos. La recomendación es crear instancias cortas y liberarlas al terminar.

```
using (var context = new AceriaDbContext())
{
    var ordenes = context.OrdenesFabricacion.ToList();
}
```

CSHARP

El bloque using asegura que el DbContext se libere al final del bloque, liberando también la conexión.

DbContext y DbSet en el proyecto AceriaData

En el proyecto AceriaData, el DbContext se llama AceriaDbContext y agrupa los DbSet de las entidades del dominio. Aleacion ya forma parte del modelo desde 1.3; en este punto se incorpora EstadoOrden y se utiliza el contexto para inspeccionar Database, Model y ChangeTracker. La configuración sigue usando SQL Server LocalDB.

Resumen de la teoría

El DbContext coordina la sesión con la base de datos.

Sus responsabilidades son: exponer DbSet, mantener el modelo, gestionar el Change Tracker y coordinar la conexión.

Expone las propiedades Database, Model, ChangeTracker y el método SaveChanges.

Un DbSet representa una tabla dentro del DbContext.

El DbContext se configura mediante DbContextOptions o OnConfiguring.

Su ciclo de vida es corto: se crea, se usa y se libera.

En el proyecto AceriaData se conserva Aleacion de 1.3 y se añade EstadoOrden.

Punto 1.5 - Ciclo de vida del DbContext en aplicaciones de consola, web y servicios

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core.

Proyecto: Se refactoriza el proyecto AceriaData para gestionar correctamente el ciclo de vida del DbContext en aplicaciones de consola, y se prepara la estructura para su uso futuro en aplicaciones web y servicios, siempre con SQL Server LocalDB.

Objetivos de aprendizaje

Comprender por qué el DbContext tiene un ciclo de vida corto.

Identificar los problemas de mantener un DbContext vivo durante mucho tiempo.

Diferenciar los patrones de ciclo de vida en aplicaciones de consola, web y servicios.

Utilizar el patrón de fábrica de DbContext en aplicaciones de consola.

Preparar el proyecto AceriaData para su uso con inyección de dependencias.

Refactorizar el código actual para crear instancias cortas y liberarlas correctamente.

Teoría

Qué es el ciclo de vida de un objeto

El ciclo de vida de un objeto es el periodo que transcurre desde que se crea hasta que se libera. En C#, los objetos gestionados por el recolector de basura no se liberan explícitamente, pero los que implementan `IDisposable` sí deben liberarse mediante `using` o una llamada explícita a `Dispose`. El `DbContext` implementa `IDisposable` porque mantiene una conexión con la base de datos y otros recursos no gestionados.

El ciclo de vida del `DbContext` debe ser corto. No está diseñado para vivir durante toda la aplicación. Está diseñado para representar una unidad de trabajo concreta: cargar datos, modificarlos, guardarlos y liberar los recursos. Este patrón se conoce como unidad de trabajo.

```
using var context = new AceriaDbContext();
var ordenes = context.OrdenesFabricacion.ToList();
```

CSHARP

En este fragmento, el `DbContext` se crea, se usa para una consulta y se libera al final del bloque `using`. Esa es la forma correcta de usarlo en una aplicación de consola.

Por qué el DbContext no debe vivir mucho tiempo

Un `DbContext` de larga duración acumula problemas. El primero es el crecimiento del `Change Tracker`: cada entidad cargada queda registrada en memoria, y el `Change Tracker` crece indefinidamente. El segundo es la caché de identidad: las entidades cargadas se almacenan en una caché que devuelve siempre la misma instancia para la misma clave, lo que puede provocar datos obsoletos. El tercero es la conexión: aunque EF Core abre y cierra la conexión según necesita, mantener el `DbContext` vivo retiene otros recursos asociados.

```
using var context = new AceriaDbContext();

for (int i = 0; i < 10000; i++)
{
    var orden = context.OrdenesFabricacion.FirstOrDefault(o => o.Id == i);
}
```

CSHARP

En este ejemplo, cada iteración carga una entidad y la deja registrada en el `Change Tracker`. Después de diez mil iteraciones, el `Change Tracker` contiene diez mil entidades en memoria. El `DbContext` consume recursos de forma innecesaria.

El patrón de unidad de trabajo

El patrón de unidad de trabajo consiste en agrupar un conjunto de operaciones relacionadas en una sola sesión con la base de datos. La sesión comienza cuando se crea el `DbContext`, agrupa las operaciones y termina cuando se llama a `SaveChanges` y se libera el contexto. Cada unidad de trabajo es independiente de las demás.

```
using (var context = new AceriaDbContext())
{
    var orden = new OrdenFabricacion { NumeroOrden = "0F-001", Cliente = "Constructora del Norte" };
    context.OrdenesFabricacion.Add(orden);
    context.SaveChanges();
}
```

CSHARP

La unidad de trabajo abarca desde la creación hasta la liberación del contexto. Todo lo que ocurre dentro es una transacción lógica. Al salir del bloque `using`, el `DbContext` se libera y la unidad de trabajo termina.

Ciclo de vida en aplicaciones de consola

En aplicaciones de consola, el patrón habitual es crear una instancia del `DbContext` por cada operación o por cada unidad de trabajo. El bloque `using` garantiza la liberación de recursos. Si la aplicación realiza varias operaciones independientes, se crea una instancia por cada una.

```
public static void InsertarOrden(string numero, string cliente)
{
    using var context = new AceriaDbContext();
    var orden = new OrdenFabricacion { NumeroOrden = numero, Cliente = cliente };
    context.OrdenesFabricacion.Add(orden);
    context.SaveChanges();
}
```

CSHARP

```
public static void ListarOrdenes()
{
    using var context = new AceriaDbContext();
    var ordenes = context.OrdenesFabricacion.ToList();
    foreach (var orden in ordenes)
    {
        Console.WriteLine($"{orden.NumeroOrden} - {orden.Cliente}");
    }
}
```

Cada método crea su propio DbContext y lo libera al terminar. Las operaciones son independientes entre sí.

Ciclo de vida en aplicaciones web

En aplicaciones web, el ciclo de vida se gestiona mediante inyección de dependencias. El DbContext se registra con ámbito por petición. Cada petición HTTP recibe una instancia nueva del DbContext, que se libera al finalizar la petición.

```
builder.Services.AddDbContext<AceriaDbContext>(options =>
    options.UseSqlServer("Server=(localdb)\\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;"));
```

CSHARP

La llamada a AddDbContext registra el DbContext con ámbito Scoped. Cada petición recibe una instancia nueva y se libera al finalizar. Este patrón evita que dos peticiones compartan el mismo DbContext.

Ciclo de vida en servicios de larga duración

En servicios de larga duración, como los BackgroundService, el DbContext no puede inyectarse directamente con ámbito Scoped porque el servicio vive más que una petición. La solución es inyectar una fábrica de DbContext (IDbContextFactory<T>) y crear instancias cortas dentro del servicio.

```
public class ProcesadorOrdenes : BackgroundService
{
    private readonly IDbContextFactory<AceriaDbContext> _factory;

    public ProcesadorOrdenes(IDbContextFactory<AceriaDbContext> factory)
    {
        _factory = factory;
    }

    protected override async Task ExecuteAsync(CancellationToken stoppingToken)
    {
        while (!stoppingToken.IsCancellationRequested)
        {
            using var context = await _factory.CreateDbContextAsync(stoppingToken);
            var pendientes = await context.OrdenesFabricacion.Where(o => o.Estado == "Pendiente").ToListAsync();
            await Task.Delay(TimeSpan.FromMinutes(1), stoppingToken);
        }
    }
}
```

CSHARP

La fábrica crea una instancia nueva del DbContext en cada iteración y la libera al final del bloque using. El servicio puede vivir indefinidamente sin acumular entidades en el Change Tracker.

La fábrica de DbContext

La interfaz IDbContextFactory<T> es el mecanismo que EF Core ofrece para crear instancias del DbContext bajo demanda. Se registra con AddDbContextFactory y se inyecta en los servicios que necesitan crear contextos de corta duración.

```
builder.Services.AddDbContextFactory<AceriaDbContext>(options =>
    options.UseSqlServer("Server=(localdb)\\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;"));
```

CSHARP

Esta línea registra la fábrica en el contenedor. Los servicios pueden recibir IDbContextFactory<AceriaDbContext> por constructor y llamar a CreateDbContextAsync cuando necesitan un contexto nuevo.

Patrón de fábrica en aplicaciones de consola

En aplicaciones de consola que no usan inyección de dependencias, se puede implementar una fábrica manual. Esta fábrica devuelve una nueva instancia del DbContext cada vez que se llama, configurada con el proveedor y la cadena de conexión.

```
public static class AceriaDbContextFactory
{
    public static AceriaDbContext Create()
    {
        return new AceriaDbContext();
    }
}
```

CSHARP

Este patrón es útil cuando se quiere centralizar la creación del DbContext sin depender del contenedor de inyección de dependencias.

Errores comunes en la gestión del ciclo de vida

Los errores más comunes son mantener el DbContext vivo durante toda la aplicación, compartirlo entre hilos y no liberarlo. El primer error provoca fugas de memoria y datos obsoletos. El segundo error provoca excepciones de concurrencia porque el DbContext no es seguro para subprocesos. El tercer error provoca que la conexión permanezca abierta más tiempo del necesario.

```
private static readonly AceriaDbContext _contextCompartido = new AceriaDbContext();
```

CSHARP

Esta línea es un anti-patrón. El DbContext estático compartido se usa desde cualquier parte de la aplicación y no se libera nunca.

El proyecto AceriaData

En el proyecto AceriaData, el DbContext se crea de forma local en cada operación. En este punto se refactoriza el código para aplicar el patrón de unidad de trabajo y prepararlo para su uso futuro con inyección de dependencias. La base de datos sigue siendo SQL Server LocalDB y el proveedor es Microsoft.EntityFrameworkCore.SqlServer.

Resumen de la teoría

El DbContext implementa IDisposable y debe liberarse al terminar.

El ciclo de vida del DbContext debe ser corto: una unidad de trabajo por instancia.

Un DbContext de larga duración acumula entidades y datos obsoletos.

En aplicaciones de consola, se crea una instancia por operación con using.

En aplicaciones web, se registra con ámbito Scoped y se inyecta.

En servicios de larga duración, se usa IDbContextFactory<T>.

El patrón de unidad de trabajo agrupa operaciones relacionadas.

En el proyecto AceriaData se refactoriza el código para aplicar este patrón.

Punto 1.6 - DbSet y operaciones básicas de acceso a datos

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core.

Proyecto: Se añaden al proyecto AceriaData las operaciones básicas de acceso a datos sobre las entidades existentes, todas contra SQL Server LocalDB.

Objetivos de aprendizaje

Comprender qué es un DbSet y cómo se usa.

Ejecutar operaciones de consulta, inserción, actualización y eliminación sobre un DbSet.

Diferenciar las operaciones que afectan al Change Tracker de las que no.

Entender el comportamiento de Add, Update, Remove y Find.

Aplicar consultas con Where, FirstOrDefault, SingleOrDefault, Any y Count.

Incorporar estas operaciones al proyecto AceriaData.

Teoría

Qué es un DbSet y cómo se construye

Un DbSet es la representación de una tabla dentro del DbContext. Cada propiedad DbSet<T> expone una colección de entidades del tipo T. Sobre un DbSet se construyen las consultas LINQ y se registran las operaciones de escritura. Un DbSet no contiene datos en memoria hasta que se materializa una consulta. Antes de eso, es una expresión que describe lo que se quiere recuperar.

La declaración de un DbSet se hace como una propiedad de solo lectura y escritura dentro del DbContext:

```
public class AceriaDbContext : DbContext
{
    public DbSet<OrdenFabricacion> OrdenesFabricacion { get; set; } = null!;
    public DbSet<PlanchaAcero> PlanchasAcero { get; set; } = null!;
    public DbSet<Aleacion> Aleaciones { get; set; } = null!;
    public DbSet<EstadoOrden> EstadosOrden { get; set; } = null!;
}
```

CSHARP

Cada propiedad expone una tabla. EF Core crea el mapeo entre el tipo T y la tabla correspondiente en la base de datos. El nombre de la tabla se deduce por convención a partir del nombre de la propiedad: OrdenesFabricacion se mapea a la tabla OrdenesFabricacion, PlanchasAcero a PlanchasAcero, y así sucesivamente. Si se quiere cambiar el nombre de la tabla, se puede hacer con Data Annotations o Fluent API, como se verá en módulos posteriores.

El DbSet no se inicializa manualmente. EF Core se encarga de crear la instancia cuando se construye el DbContext. Por eso la propiedad se declara con = null!, que indica al compilador que la propiedad no será nula en tiempo de ejecución aunque no se asigne en el constructor.

El DbSet como origen de consultas

Un DbSet implementa IQueryable<T>. Esto significa que todas las operaciones LINQ que se aplican sobre él se traducen a SQL y se ejecutan en el servidor. No se cargan datos en memoria hasta que se materializa la consulta.

```
var consulta = context.OrdenesFabricacion
    .Where(o => o.Cliente == "Constructora del Norte");

var lista = consulta.ToList();
```

CSHARP

La primera línea construye un árbol de expresión que describe la consulta. No se ejecuta nada contra la base de datos. La segunda línea materializa la consulta, momento en el que EF Core traduce el árbol de expresión a SQL, lo envía al servidor, lee los resultados y construye la lista de objetos.

Esta separación entre construcción y ejecución es fundamental. Permite componer consultas de forma incremental, añadiendo filtros, ordenaciones y proyecciones sin ejecutar nada hasta el final.

```
var consulta = context.OrdenesFabricacion.AsQueryable();

if (!string.IsNullOrEmpty(cliente))
{
    consulta = consulta.Where(o => o.Cliente == cliente);
}

if (fechaDesde.HasValue)
{
    consulta = consulta.Where(o => o.FechaCreacion >= fechaDesde.Value);
}

var resultados = consulta.ToList();
```

CSHARP

En este ejemplo, la consulta se construye de forma condicional. Solo se ejecuta un SELECT al llamar a ToList, y el SQL incluye únicamente los filtros que se aplicaron.

Inserción con Add y AddRange

El método Add registra una entidad nueva en el Change Tracker con estado Added. El INSERT no se ejecuta hasta que se llama a SaveChanges. El método AddRange permite añadir varias entidades en una sola llamada.

```
var orden = new OrdenFabricacion
{
    NumeroOrden = "OF-001",
    Cliente = "Constructora del Norte",
    FechaCreacion = DateTime.Now
};

context.OrdenesFabricacion.Add(orden);
context.SaveChanges();
```

CSHARP

La primera instrucción crea la entidad en memoria. La segunda la registra en el Change Tracker con estado Added. La tercera ejecuta el INSERT contra la base de datos, asigna el valor generado por la columna identidad a la propiedad Id y cambia el estado de la entidad a Unchanged.

El SQL generado tiene la siguiente forma:

```
INSERT INTO [OrdenesFabricacion] ([NumeroOrden], [Cliente], [FechaCreacion])
VALUES (@p0, @p1, @p2);
SELECT [Id] FROM [OrdenesFabricacion] WHERE @@ROWCOUNT = 1 AND [Id] = scope_identity();
```

SQL

La primera sentencia inserta la fila. La segunda recupera el Id generado por SQL Server. Este patrón lo genera EF Core automáticamente para las columnas identidad.

El método AddRange permite añadir varias entidades en una sola llamada. Internamente, cada entidad se registra individualmente, pero el SaveChanges posterior agrupa las inserciones en un solo lote cuando el proveedor lo soporta.

```
var plancha1 = new PlanchaAcero { OrdenId = orden.Id, Espesor = 10.5, Ancho = 1500, Largo = 3000 };
var plancha2 = new PlanchaAcero { OrdenId = orden.Id, Espesor = 12.0, Ancho = 1200, Largo = 2500 };

context.PlanchasAcero.AddRange(plancha1, plancha2);
context.SaveChanges();
```

CSHARP

El proveedor de SQL Server agrupa los dos INSERT en un solo comando cuando es posible, lo que reduce el número de viajes al servidor.

Actualización con Update y con seguimiento

Existen dos formas de actualizar una entidad en EF Core. La primera es cargarla desde la base de datos, modificar la propiedad y llamar a SaveChanges. En este caso, el Change Tracker ya conoce la entidad y detecta el cambio automáticamente.

```
var orden = context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == "OF-001");
orden!.Cliente = "Constructora del Sur";
context.SaveChanges();
```

CSHARP

La primera línea carga la entidad y la registra en el Change Tracker con estado Unchanged. La segunda modifica la propiedad Cliente. Al llamar a SaveChanges, el Change Tracker compara el valor actual con el valor original y detecta que Cliente ha cambiado. Entonces genera un UPDATE que solo actualiza esa columna.

El SQL generado tiene la siguiente forma:

```
UPDATE [OrdenesFabricacion]
SET [Cliente] = @p0
WHERE [Id] = @p1;
```

SQL

La segunda forma es usar el método Update sobre una entidad que no está siendo rastreada. En este caso, EF Core marca todas las propiedades como modificadas y genera un UPDATE con todas las columnas.

```
var orden = new OrdenFabricacion
{
    Id = 1,
    NumeroOrden = "OF-001",
```

CSHARP

```

Cliente = "Constructora del Sur",
FechaCreacion = DateTime.Now
};

context.OrdenesFabricacion.Update(orden);
context.SaveChanges();

```

El SQL generado en este caso actualiza todas las columnas:

```

UPDATE [OrdenesFabricacion]
SET [NumeroOrden] = @p0, [Cliente] = @p1, [FechaCreacion] = @p2
WHERE [Id] = @p3;

```

SQL

La diferencia entre ambos enfoques es importante. El primero genera un UPDATE mínimo. El segundo genera un UPDATE completo. En aplicaciones con muchas columnas, el primero es más eficiente.

Eliminación con Remove y RemoveRange

El método Remove marca una entidad como Deleted. El DELETE se ejecuta al llamar a SaveChanges. Si la entidad tiene relaciones configuradas en cascada, EF Core elimina también las entidades relacionadas.

```

var orden = context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == "OF-001");
context.OrdenesFabricacion.Remove(orden!);
context.SaveChanges();

```

CSHARP

La primera línea carga la entidad. La segunda la marca para eliminar. La tercera ejecuta el DELETE. El SQL generado tiene la siguiente forma:

```

DELETE FROM [OrdenesFabricacion]
WHERE [Id] = @p0;

```

SQL

Si la entidad tiene planchas asociadas y la relación está configurada con Cascade, EF Core genera también los DELETE de las planchas. Si la relación está configurada con Restrict, EF Core lanza una excepción si hay planchas asociadas.

El método RemoveRange permite eliminar varias entidades en una sola llamada.

Búsqueda por clave con Find

El método Find busca una entidad por su clave primaria. Su comportamiento es distinto al de FirstOrDefault. Primero comprueba si la entidad ya está en la caché del DbContext. Si está, la devuelve sin consultar la base de datos. Si no está, ejecuta un SELECT y la carga.

```

var orden1 = context.OrdenesFabricacion.Find(1);
var orden2 = context.OrdenesFabricacion.Find(1);

Console.WriteLine(ReferenceEquals(orden1, orden2));

```

CSHARP

La primera línea busca la entidad con Id 1. Si no está en la caché, ejecuta un SELECT. La segunda línea busca la misma entidad. Como ya está en la caché, la devuelve directamente sin consultar la base de datos. La tercera línea imprime True, confirmando que ambas variables apuntan a la misma instancia.

Este comportamiento se conoce como caché de identidad. EF Core garantiza que, dentro de un mismo DbContext, solo existe una instancia por cada entidad con una clave primaria concreta. Esto evita duplicados en memoria y asegura la coherencia de los datos.

El método Find acepta múltiples valores cuando la entidad tiene una clave compuesta.

```

var detalle = context.DetallesOrden.Find(ordenId, productoId);
Consultas con FirstOrDefault y SingleOrDefault

```

CSHARP

FirstOrDefault devuelve el primer elemento de la secuencia o null si no hay ninguno. No lanza excepción si hay varios elementos. Se traduce a SQL con SELECT TOP 1.

```

var orden = context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == "OF-001");

```

CSHARP

El SQL generado tiene la siguiente forma:

```
SELECT TOP 1 [Id], [NumeroOrden], [Cliente], [FechaCreacion]
FROM [OrdenesFabricacion]
WHERE [NumeroOrden] = @p0;
```

SQL

SingleOrDefault devuelve el único elemento de la secuencia o null si no hay ninguno. Lanza una excepción si hay más de uno. Se traduce a SQL con SELECT TOP 2 para poder detectar si hay más de un resultado.

```
var orden = context.OrdenesFabricacion.SingleOrDefault(o => o.NumeroOrden == "0F-001");
```

CSHARP

El SQL generado tiene la siguiente forma:

```
SELECT TOP 2 [Id], [NumeroOrden], [Cliente], [FechaCreacion]
FROM [OrdenesFabricacion]
WHERE [NumeroOrden] = @p0;
```

SQL

EF Core lee hasta dos filas. Si encuentra dos, lanza una excepción indicando que la secuencia contiene más de un elemento.

También existen First y Single, que lanzan excepción si no hay ningún elemento. Se usan cuando se sabe con certeza que el elemento existe.

Consultas con Any y Count

Any devuelve true si la secuencia contiene al menos un elemento. Se traduce a SQL con EXISTS, que es más eficiente que contar los elementos.

```
var existe = context.OrdenesFabricacion.Any(o => o.Cliente == "Constructora del Norte");
```

CSHARP

El SQL generado tiene la siguiente forma:

```
SELECT CASE WHEN EXISTS (
    SELECT 1 FROM [OrdenesFabricacion] WHERE [Cliente] = @p0
) THEN 1 ELSE 0 END;
```

SQL

Count devuelve el número de elementos. Se traduce a SQL con COUNT(*).

```
var total = context.OrdenesFabricacion.Count();
```

CSHARP

El SQL generado tiene la siguiente forma:

```
SELECT COUNT(*) FROM [OrdenesFabricacion];
```

SQL

Any es más eficiente que Count cuando solo se quiere saber si existe al menos un elemento. Count recorre todos los elementos, mientras que Any se detiene en el primero.

Existe también LongCount, que devuelve un long en lugar de un int, útil para tablas con más de dos mil millones de filas.

Consultas con Where, OrderBy y Select

Where filtra los elementos que cumplen una condición. Se traduce a SQL con WHERE.

```
var ordenes = context.OrdenesFabricacion
    .Where(o => o.Cliente == "Constructora del Norte")
    .ToList();
```

CSHARP

OrderBy ordena los elementos de forma ascendente. OrderByDescending los ordena de forma descendente. ThenBy y ThenByDescending permiten añadir criterios secundarios.

```
var ordenes = context.OrdenesFabricacion
    .OrderBy(o => o.Cliente)
    .ThenByDescending(o => o.FechaCreacion)
    .ToList();
```

CSHARP

Select proyecta cada elemento a una nueva forma. Puede proyectar a un tipo anónimo, a un DTO o a una propiedad concreta.

```
var clientes = context.OrdenesFabricacion
    .Select(o => o.Cliente)
    .Distinct()
    .ToList();
```

CSHARP

El SQL generado tiene la siguiente forma:

```
SELECT DISTINCT [Cliente] FROM [OrdenesFabricacion];
```

SQL

La proyección con Select es una de las técnicas más eficientes para reducir el volumen de datos transferidos. En lugar de cargar entidades completas, se cargan solo las columnas necesarias.

Operaciones que afectan al Change Tracker

Las operaciones Add, Update, Remove y Attach afectan al Change Tracker. Registran la entidad y le asignan un estado. El estado determina qué sentencia SQL se generará al llamar a SaveChanges.

Las consultas normales también afectan al Change Tracker. Cuando se carga una entidad desde la base de datos, EF Core la registra con estado Unchanged y guarda una copia de sus valores originales. Esta copia se usa para detectar cambios posteriores.

Las consultas con AsNoTracking no registran entidades en el Change Tracker. Son útiles para consultas de solo lectura en las que no se van a modificar los datos.

```
var ordenes = context.OrdenesFabricacion.AsNoTracking().ToList();
```

CSHARP

El SQL generado es el mismo que sin AsNoTracking, pero las entidades no se registran en el Change Tracker. Esto reduce el consumo de memoria y mejora el rendimiento.

La caché de identidad

EF Core realiza resolución de identidad dentro de las consultas con seguimiento: para una misma clave primaria mantiene una única instancia rastreada. Una consulta LINQ normal puede volver a ejecutarse contra la base de datos; al materializar el resultado, EF Core reutiliza la instancia ya rastreada en lugar de crear otra. El método Find tiene un comportamiento adicional: primero consulta las entidades ya rastreadas y, si encuentra la clave, puede devolverla sin enviar una nueva consulta a la base de datos.

```
var orden1 = context.OrdenesFabricacion.FirstOrDefault(o => o.Id == 1);
var orden2 = context.OrdenesFabricacion.FirstOrDefault(o => o.Id == 1);

Console.WriteLine(ReferenceEquals(orden1, orden2));
```

CSHARP

La primera línea carga la entidad con Id 1 y la registra en el Change Tracker. La segunda consulta LINQ puede volver a ejecutarse en SQL Server, pero la resolución de identidad hace que el resultado materializado reutilice la misma instancia rastreada. La tercera línea imprime True.

La caché de identidad se vacía cuando se libera el DbContext. Por eso es importante usar instancias cortas y no compartir el DbContext entre operaciones independientes.

El método Attach

El método Attach registra una entidad en el Change Tracker con estado Unchanged. Se usa cuando se quiere que EF Core empiece a rastrear una entidad que ya existe en la base de datos pero que no fue cargada por el contexto actual.

CSHARP

```
var orden = new OrdenFabricacion { Id = 1, NumeroOrden = "OF-001", Cliente = "Constructora del Norte", FechaCreacion = DateTime.Now };
context.OrdenesFabricacion.Attach(orden);
orden.Cliente = "Constructora del Sur";
context.SaveChanges();
```

La primera línea crea la entidad en memoria. La segunda la registra con estado Unchanged. La tercera modifica una propiedad. La cuarta ejecuta el UPDATE solo de la columna modificada, porque el Change Tracker detecta el cambio respecto al estado original.

El método Entry

El método Entry devuelve un objeto EntityEntry que permite inspeccionar y modificar el estado de una entidad. Expone propiedades como State, CurrentValues, OriginalValues y Property.

CSHARP

```
var orden = new OrdenFabricacion { NumeroOrden = "OF-001", Cliente = "Constructora del Norte" };
var entry = context.Entry(orden);

Console.WriteLine(entry.State);
entry.State = EntityState.Added;
Console.WriteLine(entry.State);
```

La primera línea crea la entidad. La segunda obtiene el EntityEntry. La tercera muestra el estado inicial, que es Detached. La cuarta cambia el estado a Added. La quinta muestra el nuevo estado.

El proyecto AceriaData

En el proyecto AceriaData, se añaden métodos para realizar operaciones básicas sobre las entidades existentes. Cada operación se ejecuta en su propia unidad de trabajo, con una instancia nueva del DbContext obtenida a través de la fábrica. La base de datos sigue siendo AceriaDB en SQL Server LocalDB.

Los métodos cubren las operaciones fundamentales: inserción con Add, consulta con Where y OrderBy, actualización con seguimiento, eliminación con Remove, búsqueda por clave con Find, comprobación de existencia con Any y conteo con Count. Cada método demuestra un patrón concreto que se reutilizará en los módulos posteriores.

Resumen de la teoría

Un DbSet representa una tabla dentro del DbContext y se construye como propiedad.

Las consultas se construyen con LINQ y se ejecutan al materializarse.

Add y AddRange registran entidades nuevas con estado Added.

Update marca entidades como Modified y genera un UPDATE completo.

Remove y RemoveRange marcan entidades como Deleted.

Find busca por clave primaria y usa la caché de identidad.

FirstOrDefault y SingleOrDefault devuelven una entidad o null.

Any y Count devuelven valores booleanos o numéricos.

AsNoTracking carga entidades sin registrarlas en el Change Tracker.

La caché de identidad garantiza una sola instancia por clave primaria.

Attach registra una entidad existente con estado Unchanged.

Entry permite inspeccionar y modificar el estado de una entidad.

Punto 1.7 - Change Tracker: estados de las entidades y detección de cambios

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core.

Proyecto: Se profundiza en el Change Tracker del proyecto AceriaData, observando cómo detecta cambios en las entidades y cómo se pueden inspeccionar y manipular sus estados, siempre contra SQL Server LocalDB.

Objetivos de aprendizaje

Comprender qué es el Change Tracker y cuál es su responsabilidad.

Identificar los cinco estados de una entidad: Detached, Unchanged, Added, Modified y Deleted.

Entender cómo el Change Tracker detecta cambios en las propiedades.

Utilizar Entry, Entries y ChangeTracker para inspeccionar y manipular estados.

Diferenciar entre valores actuales y valores originales.

Aplicar el Change Tracker al proyecto AceriaData.

Teoría

Qué es el Change Tracker y por qué existe

El Change Tracker es el componente interno de EF Core que se encarga de supervisar todas las entidades que el DbContext conoce. Su responsabilidad es doble: por un lado, registrar cada entidad que se carga desde la base de datos o que se añade al contexto; por otro, detectar los cambios que se producen en sus propiedades para que SaveChanges pueda generar las sentencias SQL adecuadas.

Sin el Change Tracker, EF Core no sabría qué entidades han cambiado ni qué sentencias debe ejecutar. Cuando se llama a SaveChanges, el Change Tracker recorre todas las entidades registradas, compara sus valores actuales con los originales, determina el estado de cada una y genera las sentencias SQL correspondientes. Todo este proceso es transparente para el programador, pero comprenderlo permite diagnosticar problemas y optimizar el rendimiento.

```
using var context = AceriaDbContextFactory.Create();

var orden = new OrdenFabricacion
{
    NumeroOrden = "OF-001",
    Cliente = "Constructora del Norte",
    FechaCreacion = DateTime.Now
};

context.OrdenesFabricacion.Add(orden);
context.SaveChanges();
```

CSHARP

En este fragmento, el Change Tracker registra la entidad cuando se llama a Add, le asigna el estado Added, y al llamar a SaveChanges genera un INSERT. Tras ejecutarlo, cambia el estado a Unchanged y guarda una copia de los valores actuales como valores originales.

Los cinco estados de una entidad

Cada entidad registrada en el Change Tracker tiene uno de cinco estados posibles. El estado determina qué sentencia SQL se generará al llamar a SaveChanges. Los cinco estados son Detached, Unchanged, Added, Modified y Deleted.

El estado Detached indica que la entidad no está siendo rastreada por el contexto. Es el estado inicial de cualquier entidad recién creada con new que aún no se ha añadido al contexto. Una entidad en estado Detached no se ve afectada por SaveChanges.

```
var orden = new OrdenFabricacion { NumeroOrden = "OF-001", Cliente = "Constructora del Norte" };
Console.WriteLine(context.Entry(orden).State);
```

CSHARP

La primera línea crea la entidad. La segunda línea consulta su estado a través de Entry. La salida es Detached, porque la entidad no está registrada en el contexto.

El estado Unchanged indica que la entidad está siendo rastreada y que sus valores actuales coinciden con los valores originales cargados desde la base de datos. Es el estado que tienen las entidades recién cargadas con una consulta. Una entidad en estado Unchanged no genera ninguna sentencia SQL al llamar a SaveChanges.

```
var orden = context.OrdenesFabricacion.FirstOrDefault(o => o.Id == 1);
Console.WriteLine(context.Entry(orden!).State);
```

CSHARP

La primera línea carga la entidad desde la base de datos. La segunda línea consulta su estado. La salida es Unchanged, porque la entidad se acaba de cargar y no se ha modificado.

El estado Added indica que la entidad está siendo rastreada y que se va a insertar en la base de datos en el próximo SaveChanges. Es el estado que tienen las entidades registradas con Add o AddRange. Una entidad en estado Added genera un INSERT al llamar a SaveChanges.

```
context.OrdenesFabricacion.Add(orden);
Console.WriteLine(context.Entry(orden).State);
```

CSHARP

La primera línea registra la entidad. La segunda línea consulta su estado. La salida es Added, porque la entidad se ha añadido al contexto y aún no se ha guardado.

El estado Modified indica que la entidad está siendo rastreada y que algunas de sus propiedades han cambiado respecto a los valores originales. Es el estado que adquiere una entidad cargada cuando se modifica una propiedad. Una entidad en estado Modified genera un UPDATE al llamar a SaveChanges.

```
var orden = context.OrdenesFabricacion.FirstOrDefault(o => o.Id == 1);
orden!.Cliente = "Constructora del Sur";
Console.WriteLine(context.Entry(orden).State);
```

CSHARP

La primera línea carga la entidad. La segunda modifica una propiedad. La tercera consulta su estado. La salida es Modified, porque el Change Tracker ha detectado el cambio.

El estado Deleted indica que la entidad está siendo rastreada y que se va a eliminar de la base de datos en el próximo SaveChanges. Es el estado que tienen las entidades registradas con Remove o RemoveRange. Una entidad en estado Deleted genera un DELETE al llamar a SaveChanges.

```
context.OrdenesFabricacion.Remove(orden);
Console.WriteLine(context.Entry(orden).State);
```

CSHARP

La primera línea marca la entidad para eliminar. La segunda línea consulta su estado. La salida es Deleted, porque la entidad se ha marcado para eliminar.

Cómo detecta cambios el Change Tracker

El Change Tracker detecta cambios comparando los valores actuales de las propiedades con los valores originales. Los valores originales se guardan cuando la entidad se carga desde la base de datos o cuando se guarda por primera vez. Cuando se llama a SaveChanges, el Change Tracker recorre todas las entidades rastreadas, compara sus valores y determina cuáles han cambiado.

La comparación se hace propiedad a propiedad. Si una propiedad ha cambiado, el estado de la entidad pasa a Modified. Si ninguna ha cambiado, el estado permanece en Unchanged. El Change Tracker no compara por referencia, sino por valor, usando el comparador por defecto del tipo de la propiedad.

```
var orden = context.OrdenesFabricacion.FirstOrDefault(o => o.Id == 1);
Console.WriteLine(context.Entry(orden!).State);

orden!.Cliente = "Constructora del Sur";
Console.WriteLine(context.Entry(orden).State);
```

CSHARP

La primera línea carga la entidad con estado Unchanged. La segunda consulta su estado. La tercera modifica una propiedad. La cuarta consulta el estado, que ahora es Modified.

La detección de cambios se realiza automáticamente en el momento de llamar a SaveChanges. Sin embargo, también se puede forzar manualmente con el método DetectChanges, que recorre todas las entidades rastreadas y actualiza sus estados.

CSHARP

```
orden.Cliente = "Constructora del Sur";
context.ChangeTracker.DetectChanges();
Console.WriteLine(context.Entry(orden).State);
```

La primera línea modifica la propiedad. La segunda fuerza la detección de cambios. La tercera consulta el estado, que ahora es Modified.

La propiedad Entry

La propiedad Entry del DbContext devuelve un objeto EntityEntry que permite inspeccionar y manipular el estado de una entidad concreta. Expone propiedades como State, CurrentValues, OriginalValues y Property.

CSHARP

```
var orden = context.OrdenesFabricacion.FirstOrDefault(o => o.Id == 1);
var entry = context.Entry(orden!);

Console.WriteLine($"Estado: {entry.State}");
Console.WriteLine($"Cliente actual: {entry.CurrentValues["Cliente"]}");
Console.WriteLine($"Cliente original: {entry.OriginalValues["Cliente"]}");
```

La primera línea carga la entidad. La segunda obtiene su EntityEntry. La tercera muestra el estado. La cuarta muestra el valor actual de la propiedad Cliente. La quinta muestra el valor original guardado por el Change Tracker.

La propiedad CurrentValues y OriginalValues

La propiedad CurrentValues expone los valores actuales de las propiedades de la entidad. La propiedad OriginalValues expone los valores originales, es decir, los que se cargaron desde la base de datos. La comparación entre ambos es lo que permite al Change Tracker detectar los cambios.

CSHARP

```
var orden = context.OrdenesFabricacion.FirstOrDefault(o => o.Id == 1);
orden!.Cliente = "Constructora del Sur";

var entry = context.Entry(orden);

Console.WriteLine($"Actual: {entry.CurrentValues["Cliente"]}");
Console.WriteLine($"Original: {entry.OriginalValues["Cliente"]}");
```

La primera línea carga la entidad. La segunda modifica una propiedad. La tercera obtiene el EntityEntry. La cuarta muestra el valor actual. La quinta muestra el valor original. La salida refleja que el valor actual es el nuevo y el original es el antiguo.

La propiedad Property

La propiedad Property de EntityEntry permite acceder a una propiedad concreta de la entidad y consultar si ha sido modificada, cuál es su valor actual y cuál es su valor original.

CSHARP

```
var orden = context.OrdenesFabricacion.FirstOrDefault(o => o.Id == 1);
orden!.Cliente = "Constructora del Sur";

var entry = context.Entry(orden);
var clienteProperty = entry.Property(o => o.Cliente);

Console.WriteLine($"Modificada: {clienteProperty.IsModified}");
Console.WriteLine($"Actual: {clienteProperty.CurrentValue}");
Console.WriteLine($"Original: {clienteProperty.OriginalValue}");
```

La primera línea carga la entidad. La segunda modifica la propiedad Cliente. La tercera obtiene el EntityEntry. La cuarta obtiene la propiedad Cliente. La quinta muestra si está modificada. La sexta muestra el valor actual. La séptima muestra el valor original.

La colección Entries

La propiedad Entries del Change Tracker devuelve una colección con todas las entidades rastreadas. Cada elemento de la colección es un EntityEntry que expone el estado y los valores de la entidad.

CSHARP

```
foreach (var entry in context.ChangeTracker.Entries())
{
```

```
Console.WriteLine($"{entry.Entity.GetType().Name}: {entry.State}");
}
```

El bucle recorre todas las entidades rastreadas y muestra el tipo y el estado de cada una.

También se puede filtrar por estado para obtener solo las entidades que cumplen una condición.

```
var modificadas = context.ChangeTracker.Entries()
    .Where(e => e.State == EntityState.Modified)
    .ToList();

Console.WriteLine($"Entidades modificadas: {modificadas.Count}");
```

CSHARP

La primera línea obtiene las entidades en estado Modified. La segunda muestra el número de entidades modificadas.

El método DetectChanges

El método DetectChanges fuerza al Change Tracker a recorrer todas las entidades rastreadas y actualizar sus estados. Normalmente este método se ejecuta automáticamente al llamar a SaveChanges, pero se puede forzar manualmente para inspeccionar los estados antes de guardar.

```
orden.Cliente = "Constructora del Sur";
context.ChangeTracker.DetectChanges();
```

CSHARP

La primera línea modifica una propiedad. La segunda fuerza la detección de cambios, actualizando el estado de la entidad a Modified.

El método AcceptAllChanges

El método AcceptAllChanges acepta el estado actual de las entradas rastreadas: las entidades Added y Modified pasan a Unchanged, las Deleted se desacoplan y los valores actuales pasan a ser la nueva referencia original. SaveChanges() lo llama automáticamente cuando finaliza con éxito. La llamada manual es útil cuando se usa SaveChanges(acceptAllChangesOnSuccess: false) y se quiere controlar explícitamente el momento en que se aceptan los cambios.

```
context.SaveChanges(acceptAllChangesOnSuccess: false);
context.ChangeTracker.AcceptAllChanges();
```

CSHARP

La primera línea escribe los cambios sin aceptarlos automáticamente. La segunda acepta explícitamente los cambios y actualiza el estado de seguimiento.

El método Clear

El método Clear desvincula todas las entidades del contexto. Todas las entidades pasan al estado Detached. Es útil cuando se quiere vaciar el Change Tracker para liberar memoria.

```
context.ChangeTracker.Clear();
```

CSHARP

Esta línea desvincula todas las entidades del contexto. Las entidades ya no se rastrean y no se ven afectadas por SaveChanges.

El método AsNoTracking y su relación con el Change Tracker

Las consultas con AsNoTracking cargan entidades sin registrarlas en el Change Tracker. Las entidades devueltas están en estado Detached y no se ven afectadas por SaveChanges.

```
var ordenes = context.OrdenesFabricacion.AsNoTracking().ToList();
Console.WriteLine($"Entidades rastreadas: {context.ChangeTracker.Entries().Count()}");
```

CSHARP

La primera línea carga las órdenes sin registrarlas. La segunda muestra el número de entidades rastreadas, que es cero.

El método Update y su relación con el Change Tracker

El método Update marca una entidad como Modified. Si la entidad ya está rastreada, actualiza sus valores. Si no está rastreada, la registra y la marca como Modified. Es útil para actualizar entidades desconectadas.

```
var orden = new OrdenFabricacion { Id = 1, NumeroOrden = "OF-001", Cliente = "Constructora del Sur", FechaCreacion =  
    DateTime.Now };  
context.OrdenesFabricacion.Update(orden);  
Console.WriteLine(context.Entry(orden).State);
```

La primera línea crea la entidad con valores nuevos. La segunda la marca como Modified. La tercera muestra el estado, que es Modified.

El método Attach y su relación con el Change Tracker

El método Attach registra una entidad en el Change Tracker con estado Unchanged. Se usa cuando se quiere rastrear una entidad que ya existe en la base de datos pero que no fue cargada por el contexto actual. Después de Attach, se pueden modificar propiedades y el Change Tracker detectará los cambios.

```
var orden = new OrdenFabricacion { Id = 1, NumeroOrden = "OF-001", Cliente = "Constructora del Norte", FechaCreacion  
    = DateTime.Now };  
context.OrdenesFabricacion.Attach(orden);  
Console.WriteLine(context.Entry(orden).State);  
  
orden.Cliente = "Constructora del Sur";  
context.ChangeTracker.DetectChanges();  
Console.WriteLine(context.Entry(orden).State);
```

La primera línea crea la entidad. La segunda la registra con Attach. La tercera muestra el estado, que es Unchanged. La cuarta modifica una propiedad. La quinta fuerza la detección de cambios. La sexta muestra el estado, que ahora es Modified.

El proyecto AceriaData

En el proyecto AceriaData, el Change Tracker se usa de forma implícita en todas las operaciones. En este punto se añaden métodos que inspeccionan el estado de las entidades antes y después de las operaciones, y se demuestra cómo manipular los estados manualmente cuando es necesario. La base de datos sigue siendo AceriaDB en SQL Server LocalDB.

Resumen de la teoría

El Change Tracker supervisa las entidades que el DbContext conoce.

Los cinco estados son Detached, Unchanged, Added, Modified y Deleted.

El estado determina qué sentencia SQL se genera al llamar a SaveChanges.

La detección de cambios compara valores actuales con valores originales.

Entry devuelve un EntityEntry para inspeccionar una entidad.

Entries devuelve todas las entidades rastreadas.

DetectChanges fuerza la detección de cambios.

AcceptAllChanges marca todas las entidades como Unchanged.

Clear desvincula todas las entidades.

AsNoTracking carga entidades sin registrarlas.

Update y Attach permiten trabajar con entidades desconectadas.

Punto 1.8 - Gestión de entidades: Add, Update, Remove, Attach y Entry

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core.

Proyecto: Se profundiza en la gestión de entidades del proyecto AceriaData, utilizando los métodos Add, Update, Remove, Attach y Entry sobre las entidades existentes, siempre contra SQL Server LocalDB.

Objetivos de aprendizaje

Comprender la diferencia entre entidades conectadas y desconectadas.

Utilizar Add y AddRange para registrar entidades nuevas.

Utilizar Update y UpdateRange para marcar entidades como modificadas.

Utilizar Remove y RemoveRange para marcar entidades como eliminadas.

Utilizar Attach para registrar entidades existentes sin modificar.

Utilizar Entry para manipular el estado de una entidad de forma explícita.

Aplicar estos métodos al proyecto AceriaData.

Teoría

Entidades conectadas y entidades desconectadas

En EF Core, una entidad puede estar conectada o desconectada del DbContext. Una entidad conectada está registrada en el Change Tracker y el contexto conoce su estado, sus valores actuales y sus valores originales. Una entidad desconectada no está registrada en el Change Tracker y el contexto no tiene información sobre ella.

Las entidades cargadas desde la base de datos con una consulta normal están conectadas. Las entidades creadas con new y no añadidas al contexto están desconectadas. Las entidades cargadas con AsNoTracking también están desconectadas.

```
using var context = AceriaDbContextFactory.Create();

var conectada = context.OrdenesFabricacion.FirstOrDefault(o => o.Id == 1);
var desconectada = new OrdenFabricacion { Id = 2, NumeroOrden = "0F-002", Cliente = "Constructora del Sur" };
```

CSHARP

La primera línea carga una entidad conectada. La segunda línea crea una entidad desconectada. La diferencia es que la primera está registrada en el Change Tracker y la segunda no.

La distinción entre entidades conectadas y desconectadas es fundamental para elegir el método adecuado. Add, Update, Remove y Attach se comportan de forma distinta según el estado previo de la entidad.

El método Add

El método Add registra una entidad nueva en el Change Tracker con estado Added. Si la entidad ya estaba registrada, el método lanza una excepción. Si la entidad tiene propiedades de navegación, EF Core registra también las entidades relacionadas como Added.

```
var orden = new OrdenFabricacion
{
    NumeroOrden = "0F-001",
    Cliente = "Constructora del Norte",
    FechaCreacion = DateTime.Now
};

context.OrdenesFabricacion.Add(orden);
context.SaveChanges();
```

CSHARP

La primera línea crea la entidad. La segunda la registra con estado Added. La tercera ejecuta el INSERT. Si la entidad ya estaba registrada, el método Add lanza una excepción InvalidOperationException indicando que la entidad ya está siendo rastreada.

El método Add también registra las entidades relacionadas. Si la entidad OrdenFabricacion tiene una colección de PlanchaAcero y esas planchas no están registradas, Add las registra también con estado Added.

```
var orden = new OrdenFabricacion
{
    NumeroOrden = "OF-001",
    Cliente = "Constructora del Norte",
    FechaCreacion = DateTime.Now,
    Planchas = new List<PlanchaAcero>
    {
        new PlanchaAcero { Espesor = 10.5, Ancho = 1500, Largo = 3000 },
        new PlanchaAcero { Espesor = 12.0, Ancho = 1200, Largo = 2500 }
    }
};

context.OrdenesFabricacion.Add(orden);
context.SaveChanges();
```

CSHARP

En este ejemplo, las dos planchas se registran como Added junto con la orden. Al llamar a SaveChanges, EF Core inserta la orden, obtiene su Id y lo propaga a las planchas como clave foránea.

El método AddRange

El método AddRange permite añadir varias entidades en una sola llamada. Internamente, cada entidad se registra individualmente con estado Added. El SaveChanges posterior agrupa las inserciones en un solo lote cuando el proveedor lo soporta.

```
var orden1 = new OrdenFabricacion { NumeroOrden = "OF-001", Cliente = "Constructora del Norte", FechaCreacion =
DateTime.Now };
var orden2 = new OrdenFabricacion { NumeroOrden = "OF-002", Cliente = "Constructora del Sur", FechaCreacion =
DateTime.Now };

context.OrdenesFabricacion.AddRange(orden1, orden2);
context.SaveChanges();
```

CSHARP

La primera línea crea la primera orden. La segunda crea la segunda. La tercera registra ambas con estado Added. La cuarta ejecuta los dos INSERT en un solo comando cuando es posible.

El método Update

El método Update marca una entidad como Modified. Su comportamiento depende del estado previo de la entidad. Si la entidad está desconectada, Update la registra y marca todas sus propiedades como modificadas. Si la entidad ya está conectada y en estado Unchanged, Update no cambia nada. Si la entidad está conectada y en estado Modified, Update mantiene el estado.

```
var orden = new OrdenFabricacion
{
    Id = 1,
    NumeroOrden = "OF-001",
    Cliente = "Constructora del Norte Actualizada",
    FechaCreacion = DateTime.Now
};

context.OrdenesFabricacion.Update(orden);
context.SaveChanges();
```

CSHARP

La primera línea crea la entidad desconectada. La segunda la marca como Modified. La tercera ejecuta el UPDATE. El SQL generado actualiza todas las columnas, no solo las que han cambiado.

```
UPDATE [OrdenesFabricacion]
SET [NumeroOrden] = @p0, [Cliente] = @p1, [FechaCreacion] = @p2
WHERE [Id] = @p3;
```

SQL

El método Update es útil cuando se recibe una entidad desde una fuente externa, como una API, y se quiere actualizar todos sus valores en la base de datos. Sin embargo, genera un UPDATE completo, lo que puede ser ineficiente si solo ha cambiado una columna.

El método UpdateRange

El método UpdateRange permite marcar varias entidades como Modified en una sola llamada. Al igual que AddRange, internamente registra cada entidad individualmente.

```
var orden1 = new OrdenFabricacion { Id = 1, NumeroOrden = "OF-001", Cliente = "Constructora del Norte Actualizada",
FechaCreacion = DateTime.Now };
var orden2 = new OrdenFabricacion { Id = 2, NumeroOrden = "OF-002", Cliente = "Constructora del Sur Actualizada",
FechaCreacion = DateTime.Now };

context.OrdenesFabricacion.UpdateRange(orden1, orden2);
context.SaveChanges();
```

La primera línea crea la primera orden. La segunda crea la segunda. La tercera marca ambas como Modified. La cuarta ejecuta los dos UPDATE.

El método Remove

El método Remove marca una entidad como Deleted. El comportamiento depende del estado previo de la entidad. Si la entidad está conectada, Remove cambia su estado a Deleted. Si la entidad está desconectada, Remove la registra y la marca como Deleted. Si la entidad tiene relaciones configuradas en cascada, EF Core elimina también las entidades relacionadas.

```
var orden = context.OrdenesFabricacion.FirstOrDefault(o => o.Id == 1);
context.OrdenesFabricacion.Remove(orden!);
context.SaveChanges();
```

La primera línea carga la entidad con estado Unchanged. La segunda la marca como Deleted. La tercera ejecuta el DELETE. El SQL generado elimina la fila correspondiente.

```
DELETE FROM [OrdenesFabricacion]
WHERE [Id] = @p0;
```

Si la entidad tiene planchas asociadas y la relación está configurada con Cascade, EF Core genera también los DELETE de las planchas. Si la relación está configurada con Restrict, EF Core lanza una excepción si hay planchas asociadas.

El método RemoveRange

El método RemoveRange permite marcar varias entidades como Deleted en una sola llamada.

```
var ordenes = context.OrdenesFabricacion.Where(o => o.Cliente == "Constructora del Sur").ToList();
context.OrdenesFabricacion.RemoveRange(ordenes);
context.SaveChanges();
```

La primera línea carga las órdenes del cliente indicado. La segunda las marca todas como Deleted. La tercera ejecuta los DELETE.

El método Attach

El método Attach registra una entidad en el Change Tracker con estado Unchanged. Se usa cuando se quiere rastrear una entidad que ya existe en la base de datos pero que no fue cargada por el contexto actual. Después de Attach, se pueden modificar propiedades y el Change Tracker detectará los cambios.

```
var orden = new OrdenFabricacion
{
    Id = 1,
    NumeroOrden = "OF-001",
    Cliente = "Constructora del Norte",
    FechaCreacion = DateTime.Now
};

context.OrdenesFabricacion.Attach(orden);
Console.WriteLine(context.Entry(orden).State);

orden.Cliente = "Constructora del Sur";
```

```
context.ChangeTracker.DetectChanges();
Console.WriteLine(context.Entry(orden).State);
```

La primera línea crea la entidad desconectada. La segunda la registra con estado Unchanged. La tercera muestra el estado. La cuarta modifica una propiedad. La quinta fuerza la detección de cambios. La sexta muestra el estado, que ahora es Modified.

El SQL generado en este caso actualiza solo la columna modificada:

```
UPDATE [OrdenesFabricacion]
SET [Cliente] = @p0
WHERE [Id] = @p1;
```

SQL

La diferencia con Update es importante. Attach registra la entidad como Unchanged y solo detecta los cambios que se produzcan después. Update marca todas las propiedades como modificadas y genera un UPDATE completo.

El método Entry

El método Entry devuelve un objeto EntityEntry que permite inspeccionar y manipular el estado de una entidad. Expone propiedades como State, CurrentValues, OriginalValues y Property. También permite cambiar el estado manualmente.

```
var orden = new OrdenFabricacion
{
    Id = 1,
    NumeroOrden = "OF-001",
    Cliente = "Constructora del Norte",
    FechaCreacion = DateTime.Now
};

var entry = context.Entry(orden);
Console.WriteLine(entry.State);

entry.State = EntityState.Modified;
Console.WriteLine(entry.State);
```

CSHARP

La primera línea crea la entidad. La segunda obtiene el EntityEntry. La tercera muestra el estado inicial, que es Detached. La cuarta cambia el estado a Modified. La quinta muestra el nuevo estado.

El método Entry también permite acceder a los valores de las propiedades y modificarlos sin modificar la entidad directamente.

```
var entry = context.Entry(orden);
entry.CurrentValues["Cliente"] = "Constructora del Sur";
entry.State = EntityState.Modified;
context.SaveChanges();
```

CSHARP

La primera línea obtiene el EntityEntry. La segunda modifica el valor de la propiedad Cliente a través de CurrentValues. La tercera marca la entidad como Modified. La cuarta ejecuta el UPDATE.

El método Entry con propiedades individuales

La propiedad Property de EntityEntry permite acceder a una propiedad concreta y consultar si ha sido modificada, cuál es su valor actual y cuál es su valor original. También permite marcar una propiedad como modificada sin modificar el estado de toda la entidad.

```
var orden = context.OrdenesFabricacion.FirstOrDefault(o => o.Id == 1!);
var entry = context.Entry(orden);

entry.Property(o => o.Cliente).CurrentValue = "Constructora del Sur";
entry.Property(o => o.Cliente).IsModified = true;

context.SaveChanges();
```

CSHARP

La primera línea carga la entidad. La segunda obtiene el EntityEntry. La tercera modifica el valor de la propiedad Cliente. La cuarta marca la propiedad como modificada. La quinta ejecuta el UPDATE, que solo actualiza la columna Cliente.

Diferencias entre Update y Attach

La diferencia principal entre Update y Attach es el estado inicial que asignan a la entidad. Update marca todas las propiedades como modificadas y genera un UPDATE completo. Attach registra la entidad como Unchanged y solo detecta los cambios que se produzcan después.

```
var orden = new OrdenFabricacion
{
    Id = 1,
    NumeroOrden = "OF-001",
    Cliente = "Constructora del Norte Actualizada",
    FechaCreacion = DateTime.Now
};

context.OrdenesFabricacion.Update(orden);
context.SaveChanges();
```

CSHARP

Con Update, el SQL actualiza todas las columnas, aunque solo haya cambiado una.

```
var orden = new OrdenFabricacion
{
    Id = 1,
    NumeroOrden = "OF-001",
    Cliente = "Constructora del Norte",
    FechaCreacion = DateTime.Now
};

context.OrdenesFabricacion.Attach(orden);
orden.Cliente = "Constructora del Norte Actualizada";
context.SaveChanges();
```

CSHARP

Con Attach, el SQL actualiza solo la columna modificada, porque el Change Tracker detecta el cambio comparando el valor actual con el original.

Diferencias entre Add y Attach

La diferencia principal entre Add y Attach es el estado que asignan a la entidad. Add marca la entidad como Added y genera un INSERT. Attach marca la entidad como Unchanged y no genera ninguna sentencia SQL hasta que se modifique alguna propiedad.

```
var orden = new OrdenFabricacion { NumeroOrden = "OF-001", Cliente = "Constructora del Norte", FechaCreacion =
DateTime.Now };
context.OrdenesFabricacion.Add(orden);
context.SaveChanges();
```

CSHARP

Con Add, el SQL inserta una fila nueva.

```
var orden = new OrdenFabricacion { Id = 1, NumeroOrden = "OF-001", Cliente = "Constructora del Norte", FechaCreacion
= DateTime.Now };
context.OrdenesFabricacion.Attach(orden);
context.SaveChanges();
```

CSHARP

Con Attach, no se ejecuta ninguna sentencia SQL porque la entidad está en estado Unchanged. Si se modifica una propiedad después de Attach, se genera un UPDATE.

El proyecto AceriaData

En el proyecto AceriaData, se añaden métodos que demuestran el comportamiento de cada uno de estos métodos sobre las entidades existentes. Se comparan los SQL generados por Update y Attach, se demuestra el uso de Entry para manipular estados y se aplican estos métodos a las órdenes y planchas del dominio. La base de datos sigue siendo AceriaDB en SQL Server LocalDB.

Resumen de la teoría

Las entidades pueden estar conectadas o desconectadas del DbContext.

Add registra entidades nuevas con estado Added.

AddRange registra varias entidades en una sola llamada.

Update marca entidades como Modified y genera un UPDATE completo.

UpdateRange marca varias entidades como Modified.

Remove marca entidades como Deleted.

RemoveRange marca varias entidades como Deleted.

Attach registra entidades existentes con estado Unchanged.

Entry permite inspeccionar y manipular estados manualmente.

Property permite marcar propiedades individuales como modificadas.

Update genera un UPDATE completo; Attach solo actualiza las columnas modificadas.

Punto 1.9 - SaveChanges y unidad de trabajo

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core.

Proyecto: Se profundiza en el método SaveChanges del proyecto AceriaData, observando su comportamiento interno, el uso de transacciones implícitas y la aplicación del patrón de unidad de trabajo, siempre contra SQL Server LocalDB.

Objetivos de aprendizaje

Comprender qué hace SaveChanges internamente.

Entender cómo SaveChanges recorre el Change Tracker y genera las sentencias SQL.

Identificar el comportamiento transaccional de SaveChanges.

Conocer la diferencia entre SaveChanges y SaveChangesAsync.

Aplicar el patrón de unidad de trabajo con SaveChanges.

Gestionar errores y conflictos de concurrencia durante SaveChanges.

Incorporar estos conceptos al proyecto AceriaData.

Teoría

Qué es SaveChanges

SaveChanges es el método del DbContext que materializa en la base de datos todos los cambios registrados en el Change Tracker. Hasta que no se llama a SaveChanges, las modificaciones realizadas sobre las entidades existen únicamente en memoria. SaveChanges es el punto en el que EF Core traduce esas modificaciones a sentencias SQL, las ejecuta y actualiza el estado de las entidades.

```
using var context = AceriaDbContextFactory.Create();

var orden = new OrdenFabricacion
{
    NumeroOrden = "OF-001",
    Cliente = "Constructora del Norte",
    FechaCreacion = DateTime.Now
};

context.OrdenesFabricacion.Add(orden);
context.SaveChanges();
```

CSHARP

La primera línea crea el contexto. La segunda crea la entidad. La tercera la registra con estado Added. La cuarta ejecuta el INSERT contra la base de datos. Si no se llamara a SaveChanges, la entidad existiría solo en memoria y se perdería al liberar el contexto.

SaveChanges devuelve un entero que indica el número de entradas de estado escritas en la base de datos. No debe interpretarse de forma general como un contador exacto de filas SQL afectadas, porque una entrada puede implicar más de una operación dependiendo del modelo y del proveedor.

CSHARP

```
var entradasEscritas = context.SaveChanges();
Console.WriteLine($"Entradas escritas: {entradasEscritas}");
```

La primera línea guarda los cambios y devuelve el número de entradas de estado escritas. La segunda muestra ese número. En un ejemplo simple con dos entidades insertadas y una actualizada, el valor normalmente será tres.

El proceso interno de SaveChanges

Cuando se llama a `SaveChanges`, EF Core ejecuta una secuencia de pasos. Primero, invoca `DetectChanges` cuando la detección automática está habilitada. Después recopila las entradas que deben persistirse, ordena los comandos según las dependencias y ejecuta las operaciones mediante el proveedor. Si una única llamada requiere varias operaciones y el proveedor admite transacciones, EF Core garantiza la atomicidad de esa llamada. Con el comportamiento predeterminado `AutoTransactionBehavior.WhenNeeded`, EF crea una transacción explícita sólo cuando es necesaria; una única sentencia puede apoyarse en la atomicidad propia de la base de datos sin emitir `BEGIN TRANSACTION`. Tras un guardado correcto, `SaveChanges()` acepta los cambios por defecto y actualiza los estados de seguimiento.

CSHARP

```
var orden = new OrdenFabricacion { NumeroOrden = "OF-001", Cliente = "Constructora del Norte", FechaCreacion =
    DateTime.Now };
var plancha = new PlanchaAcero { Espesor = 10.5, Ancho = 1500, Largo = 3000, Orden = orden };

context.OrdenesFabricacion.Add(orden);
context.PlanchasAcero.Add(plancha);
context.SaveChanges();
```

En este ejemplo, EF Core detecta que plancha depende de orden. Por eso inserta primero la orden, obtiene su `Id` generado y después inserta la plancha con la clave foránea correspondiente. El orden de las operaciones lo determina EF Core automáticamente.

El comportamiento transaccional de SaveChanges

De forma predeterminada, una única llamada a `SaveChanges` es atómica cuando el proveedor admite transacciones: si una de las operaciones necesarias falla, los cambios de esa llamada se revierten. EF Core crea una transacción explícita cuando la necesita; no debe asumirse que cada `SaveChanges` emite siempre `BEGIN TRANSACTION` y `COMMIT`.

CSHARP

```
try
{
    context.OrdenesFabricacion.Add(orden);
    context.PlanchasAcero.Add(plancha);
    context.SaveChanges();
}
catch (DbUpdateException ex)
{
    Console.WriteLine($"Error al guardar: {ex.Message}");
}
```

En este ejemplo, si la inserción de la plancha falla por una clave foránea inválida, la inserción de la orden también se revierte. La base de datos no queda en un estado intermedio.

Cuando EF Core necesita una transacción automática, la crea y la confirma o revierte según el resultado. Para la mayoría de aplicaciones no es necesario controlar transacciones manualmente, pero una llamada que se traduzca en una sola sentencia puede no mostrar una transacción explícita en el log.

La propiedad Database.CurrentTransaction

La propiedad `Database.CurrentTransaction` expone la transacción actual, si existe. Es útil para diagnosticar si una operación está dentro de una transacción explícita o implícita.

CSHARP

```
var transaccion = context.Database.CurrentTransaction;
Console.WriteLine(transaccion is null ? "Sin transacción explícita" : "Transacción activa");
```

La primera línea obtiene la transacción actual. La segunda muestra si existe o no. Durante un `SaveChanges`, la transacción está activa, pero fuera de él es `null` salvo que se haya iniciado una transacción explícita.

El método SaveChangesAsync

SaveChangesAsync es la versión asíncrona de SaveChanges. Ejecuta las mismas operaciones pero devuelve un Task<int> en lugar de un int. Se usa en aplicaciones que necesitan liberar el hilo mientras se espera la respuesta de la base de datos.

```
var filasAfectadas = await context.SaveChangesAsync();
Console.WriteLine($"Filas afectadas: {filasAfectadas}");
```

CSHARP

La primera línea guarda los cambios de forma asíncrona y espera el resultado. La segunda muestra el número de filas afectadas. En aplicaciones de consola, el método Main debe ser async Task para poder usar await.

La diferencia entre ambos métodos es el modelo de ejecución, no el resultado. SaveChangesAsync no bloquea el hilo durante la operación, lo que mejora la escalabilidad en aplicaciones web y servicios.

El método ChangeTracker.AutoDetectChangesEnabled

La propiedad AutoDetectChangesEnabled del Change Tracker controla si la detección de cambios se ejecuta automáticamente antes de SaveChanges. Por defecto está en true. Desactivarla puede mejorar el rendimiento en escenarios de carga masiva, pero obliga a llamar a DetectChanges manualmente.

```
context.ChangeTracker.AutoDetectChangesEnabled = false;

foreach (var orden in ordenes)
{
    orden.Cliente = "Constructora del Norte Modificada";
}

context.ChangeTracker.DetectChanges();
context.SaveChanges();
```

CSHARP

La primera línea desactiva la detección automática. El bucle modifica varias entidades. La llamada a DetectChanges fuerza la detección manual. La llamada a SaveChanges guarda los cambios. Este patrón es útil cuando se modifican muchas entidades y se quiere controlar cuándo se ejecuta la detección.

El patrón de unidad de trabajo

El patrón de unidad de trabajo consiste en agrupar un conjunto de operaciones relacionadas en una sola transacción. En EF Core, la unidad de trabajo se implementa con una instancia de DbContext que agrupa varias operaciones y se guarda con una sola llamada a SaveChanges.

```
using var context = AceriaDbContextFactory.Create();

var orden = new OrdenFabricacion { NumeroOrden = "OF-001", Cliente = "Constructora del Norte", FechaCreacion =
DateTime.Now };
context.OrdenesFabricacion.Add(orden);

var plancha1 = new PlanchaAcero { Espesor = 10.5, Ancho = 1500, Largo = 3000, Orden = orden };
var plancha2 = new PlanchaAcero { Espesor = 12.0, Ancho = 1200, Largo = 2500, Orden = orden };
context.PlanchasAcero.AddRange(plancha1, plancha2);

context.SaveChanges();
```

CSHARP

En este ejemplo, la orden y las dos planchas forman una unidad de trabajo. Las tres se insertan en una sola transacción. Si alguna falla, ninguna se guarda.

El patrón de unidad de trabajo aporta varias ventajas. Garantiza la coherencia de los datos. Reduce el número de viajes a la base de datos. Permite agrupar operaciones relacionadas. Y facilita el testing al poder simular una unidad de trabajo completa.

La relación entre unidad de trabajo y DbContext

El DbContext es la implementación natural del patrón de unidad de trabajo en EF Core. Cada instancia de DbContext representa una unidad de trabajo. Las operaciones se acumulan en el Change Tracker y se materializan con una sola llamada a SaveChanges.

```
using (var context = AceriaDbContextFactory.Create())
{
```

CSHARP

```

var orden = new OrdenFabricacion { NumeroOrden = "OF-001", Cliente = "Constructora del Norte", FechaCreacion =
DateTime.Now };
context.OrdenesFabricacion.Add(orden);
context.SaveChanges();
}

using (var context = AceriaDbContextFactory.Create())
{
    var orden = context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == "OF-001");
    orden!.Cliente = "Constructora del Sur";
    context.SaveChanges();
}

```

En este ejemplo, cada bloque using representa una unidad de trabajo independiente. La primera inserta una orden. La segunda la actualiza. Las dos operaciones son independientes y cada una tiene su propia transacción.

El método SaveChanges y las relaciones

Cuando se guardan entidades relacionadas, EF Core ordena las operaciones según las dependencias. Primero inserta las entidades principales, después las dependientes. Para las actualizaciones, el orden es el inverso: primero las dependientes, después las principales. Para las eliminaciones, el orden depende de la configuración de la relación.

```

var orden = new OrdenFabricacion
{
    NumeroOrden = "OF-001",
    Cliente = "Constructora del Norte",
    FechaCreacion = DateTime.Now,
    Planchas = new List<PlanchaAcero>
    {
        new PlanchaAcero { Espesor = 10.5, Ancho = 1500, Largo = 3000 },
        new PlanchaAcero { Espesor = 12.0, Ancho = 1200, Largo = 2500 }
    }
};

context.OrdenesFabricacion.Add(orden);
context.SaveChanges();

```

CSHARP

La primera línea crea la orden con dos planchas relacionadas. La segunda registra la orden y, por propagación, las planchas. La tercera inserta primero la orden y después las planchas, propagando la clave foránea generada.

Si la relación está configurada con Cascade, eliminar la orden elimina también las planchas. Si está configurada con Restrict, EF Core lanza una excepción si hay planchas asociadas.

El método SaveChanges y las transacciones explícitas

Además de la transacción implícita de SaveChanges, EF Core permite iniciar transacciones explícitas que abarcan varias llamadas a SaveChanges. Esto es útil cuando se quiere agrupar varias unidades de trabajo en una sola transacción.

```

using var transaction = context.Database.BeginTransaction();

try
{
    context.OrdenesFabricacion.Add(orden1);
    context.SaveChanges();

    context.OrdenesFabricacion.Add(orden2);
    context.SaveChanges();

    transaction.Commit();
}
catch
{
    transaction.Rollback();
    throw;
}

```

CSHARP

La primera línea inicia la transacción. El bloque try ejecuta dos unidades de trabajo. La llamada a Commit confirma la transacción. La llamada a Rollback la revierte en caso de error.

El método SaveChanges y la concurrencia

Cuando una operación está configurada con control de concurrencia, EF Core incluye el valor original del token de concurrencia en la condición de actualización o eliminación. Si la operación afecta a un número inesperado de filas, puede lanzar `DbUpdateConcurrencyException`. Una modificación concurrente por sí sola no se detecta mágicamente si todavía no existe un token o una condición de concurrencia configurada; ese mecanismo se desarrollará en el Módulo 5.

```
try
{
    context.SaveChanges();
}
catch (DbUpdateConcurrencyException ex)
{
    Console.WriteLine($"Conflicto de concurrencia: {ex.Message}");
}
```

CSHARP

La primera línea intenta guardar los cambios. Si hay un conflicto, se lanza la excepción. La segunda línea captura la excepción y muestra el mensaje.

El método SaveChanges y los errores de validación

EF Core no realiza validaciones de negocio. Solo valida las restricciones de la base de datos: claves primarias, claves foráneas, restricciones de unicidad y restricciones de comprobación. Si una operación viola alguna de estas restricciones, se lanza una excepción `DbUpdateException`.

```
try
{
    context.OrdenesFabricacion.Add(ordenDuplicada);
    context.SaveChanges();
}
catch (DbUpdateException ex)
{
    Console.WriteLine($"Error de base de datos: {ex.InnerException?.Message}");
}
```

CSHARP

La primera línea añade una orden con un número duplicado. La segunda intenta guardar. La tercera captura la excepción. La cuarta muestra el mensaje de la excepción interna, que contiene el detalle del error de SQL Server.

El método SaveChanges y la propagación de claves

Cuando se inserta una entidad con una clave primaria generada por la base de datos, EF Core recupera el valor generado y lo asigna a la propiedad correspondiente. Este valor se propaga a las entidades relacionadas que lo necesiten como clave foránea.

```
var orden = new OrdenFabricacion { NumeroOrden = "OF-001", Cliente = "Constructora del Norte", FechaCreacion =
DateTime.Now };
var plancha = new PlanchaAcero { Espesor = 10.5, Ancho = 1500, Largo = 3000, Orden = orden };

context.OrdenesFabricacion.Add(orden);
context.PlanchasAcero.Add(plancha);
context.SaveChanges();

Console.WriteLine($"Orden Id: {orden.Id}");
Console.WriteLine($"Plancha OrdenId: {plancha.OrdenId}");
```

CSHARP

La primera línea crea la orden. La segunda crea la plancha y la relaciona con la orden. La tercera y cuarta registran ambas entidades. La quinta ejecuta los INSERT. Después del `SaveChanges`, la orden tiene su `Id` generado y la plancha tiene la clave foránea correspondiente.

El método SaveChanges en el proyecto AceriaData

En el proyecto `AceriaData`, `SaveChanges` se usa en todas las operaciones de escritura. En este punto se añaden métodos que demuestran su comportamiento transaccional, la propagación de claves, el manejo de errores y la aplicación del patrón de unidad de trabajo. La base de datos sigue siendo `AceriaDB` en SQL Server LocalDB.

Resumen de la teoría

`SaveChanges` materializa en la base de datos los cambios del `Change Tracker`.

Devuelve el número de filas afectadas.

Internamente ejecuta DetectChanges, ordena las operaciones y las envuelve en una transacción.

La transacción es implícita y se confirma o revierte automáticamente.

SaveChangesAsync es la versión asíncrona.

AutoDetectChangesEnabled controla la detección automática de cambios.

El patrón de unidad de trabajo agrupa operaciones relacionadas.

Las claves generadas se propagan a las entidades relacionadas.

Los conflictos de concurrencia lanzan DbUpdateConcurrencyException.

Los errores de base de datos lanzan DbUpdateException.

Punto 1.10 - Configuración inicial: opciones, cadena de conexión y logging

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core.

Proyecto: Se profundiza en la configuración del DbContext del proyecto AceriaData, explorando las opciones, la cadena de conexión y el logging de EF Core, siempre contra SQL Server LocalDB.

Objetivos de aprendizaje

Comprender cómo se configura el DbContext mediante DbContextOptions.

Diferenciar entre configurar el DbContext en OnConfiguring y por constructor.

Conocer los componentes de una cadena de conexión de SQL Server.

Almacenar la cadena de conexión en un archivo de configuración externo.

Entender el sistema de logging de EF Core.

Configurar el nivel de logging y los mensajes registrados.

Filtrar los mensajes de logging por categoría y por nivel.

Aplicar estas configuraciones al proyecto AceriaData.

Teoría

Qué es DbContextOptions

DbContextOptions es el objeto que agrupa todas las opciones de configuración del DbContext. Contiene el proveedor de base de datos, la cadena de conexión, el logging, el comportamiento de seguimiento de cambios, la sensibilidad a mayúsculas, el tiempo de espera de comandos y otras opciones. Las opciones se construyen con DbContextOptionsBuilder y se pasan al DbContext en el momento de su creación.

```
var options = new DbContextOptionsBuilder<AceriaDbContext>()
    .UseSqlServer("Server=(localdb)\\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;")
    .Options;

using var context = new AceriaDbContext(options);
```

CSHARP

La primera línea crea el constructor de opciones. La segunda registra el proveedor de SQL Server con la cadena de conexión. La tercera obtiene el objeto DbContextOptions construido. La cuarta crea el DbContext pasándole las opciones por constructor.

Este patrón es el recomendado en aplicaciones que usan inyección de dependencias. El DbContext no sabe cómo configurarse a sí mismo: recibe las opciones desde fuera.

Configuración por constructor

Cuando el DbContext recibe las opciones por constructor, necesita un constructor que las acepte. El constructor pasa las opciones a la clase base DbContext.

```
public class AceriaDbContext : DbContext
{
    public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }

    public DbSet<OrdenFabricacion> OrdenesFabricacion { get; set; } = null!;
}
```

CSHARP

La primera línea declara el constructor. La segunda pasa las opciones a la clase base. Este patrón permite que el DbContext sea configurado por el contenedor de inyección de dependencias o por el código que lo crea.

Configuración en OnConfiguring

Cuando el DbContext no recibe opciones por constructor, se puede configurar en el método OnConfiguring. Este método se ejecuta una vez por cada instancia del DbContext y recibe un DbContextOptionsBuilder que se puede configurar.

```
public class AceriaDbContext : DbContext
{
    public DbSet<OrdenFabricacion> OrdenesFabricacion { get; set; } = null!;

    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
        optionsBuilder.UseSqlServer(
            "Server=(localdb)\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;");
    }
}
```

CSHARP

La primera línea declara el método. La segunda configura el proveedor de SQL Server con la cadena de conexión. Este patrón es útil en aplicaciones de consola que no usan inyección de dependencias.

La condición IsConfigured

Cuando un DbContext puede recibir opciones por constructor o configurarse en OnConfiguring, es habitual comprobar si las opciones ya están configuradas antes de aplicar la configuración por defecto. La propiedad IsConfigured indica si el DbContextOptionsBuilder ya tiene un proveedor registrado.

```
protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
{
    if (!optionsBuilder.IsConfigured)
    {
        optionsBuilder.UseSqlServer(
            "Server=(localdb)\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;");
    }
}
```

CSHARP

La primera línea declara el método. La segunda comprueba si las opciones ya están configuradas. La tercera configura el proveedor solo si no lo estaban. Este patrón evita sobrescribir la configuración cuando el DbContext recibe opciones por constructor.

Componentes de una cadena de conexión

Una cadena de conexión de SQL Server contiene varios componentes separados por punto y coma. Los más habituales son Server, Database, Trusted_Connection, User Id y Password.

```
Server=(localdb)\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;
```

TEXT

Server → indica la instancia de SQL Server. (localdb)\mssqllocaldb es la instancia por defecto de LocalDB.

Database → nombre de la base de datos. Si no existe, EF Core la crea cuando se llama a EnsureCreated o se aplican migraciones.

Trusted_Connection=True → usa la autenticación de Windows del usuario actual. Es equivalente a Integrated Security=True.

Otras opciones habituales son MultipleActiveResultSets, Encrypt, TrustServerCertificate y Connect Timeout.

```
Server=(localdb)\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;MultipleActiveResultSets=true;Connect Timeout=30;
```

TEXT

MultipleActiveResultSets=true → permite tener varios lectores activos en la misma conexión.
Connect Timeout=30 → establece el tiempo máximo de espera para establecer la conexión, en segundos.

Alternativas a Trusted_Connection

Cuando se usa autenticación de SQL Server en lugar de autenticación de Windows, se especifican User Id y Password en lugar de Trusted_Connection.

```
Server=localhost;Database=AceriaDB;User Id=sa;Password=MíContraseña123;TrustServerCertificate=True;
```

TEXT

User Id=sa → nombre de usuario de SQL Server.

Password=MíContraseña123 → contraseña del usuario.

TrustServerCertificate=True → acepta el certificado del servidor sin validarlo. Es útil en entornos de desarrollo con certificados autofirmados.

En LocalDB no se usa autenticación de SQL Server porque se ejecuta en modo usuario y usa la autenticación de Windows.

Almacenar la cadena de conexión en un archivo de configuración

En aplicaciones .NET, la cadena de conexión se almacena habitualmente en un archivo de configuración externo. El archivo appsettings.json es el formato estándar en .NET moderno. Para leerlo, se necesita el paquete Microsoft.Extensions.Configuration.Json.

```
{
  "ConnectionStrings": {
    "AceriaDB": "Server=(localdb)\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;"
  }
}
```

JSON

La propiedad ConnectionStrings agrupa las cadenas de conexión. La clave AceriaDB identifica la cadena concreta. Para leerla desde el código, se usa el método GetConnectionString.

```
var configuration = new ConfigurationBuilder()
    .SetBasePath(Directory.GetCurrentDirectory())
    .AddJsonFile("appsettings.json")
    .Build();

var connectionString = configuration.GetConnectionString("AceriaDB");
```

CSHARP

La primera línea crea el constructor de configuración. La segunda establece el directorio base. La tercera añade el archivo appsettings.json. La cuarta construye la configuración. La quinta lee la cadena de conexión por su clave.

Variables de entorno

Las variables de entorno son otra forma de almacenar la cadena de conexión. Son útiles en entornos de producción donde no se quiere incluir la cadena en el código ni en archivos de configuración versionados.

```
export ConnectionStrings__AceriaDB="Server=(localdb)\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;"
```

BASH

La variable ConnectionStrings__AceriaDB usa el doble guion bajo para representar la jerarquía de la configuración. El proveedor de variables de entorno la mapea a ConnectionStrings:AceriaDB.

```
var configuration = new ConfigurationBuilder()
    .AddJsonFile("appsettings.json")
    .AddEnvironmentVariables()
    .Build();
```

CSHARP

La primera línea crea el constructor. La segunda añade el archivo JSON. La tercera añade las variables de entorno. La cuarta construye la configuración. Las variables de entorno sobrescriben los valores del archivo JSON cuando tienen la misma clave.

Qué es el logging en EF Core

El logging es el mecanismo que EF Core usa para registrar información sobre las operaciones que realiza. Los mensajes incluyen las sentencias SQL generadas, los parámetros, los tiempos de ejecución, las transacciones y los errores. El logging es fundamental para diagnosticar problemas de rendimiento y para entender qué está haciendo EF Core internamente.

```
optionsBuilder
    .UseSqlServer("Server=(localdb)\\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;")
    .LogTo(Console.WriteLine);
```

CSHARP

La primera línea configura el proveedor de SQL Server. La segunda habilita el logging en la consola. Todos los mensajes de EF Core se escriben en la salida estándar.

El método LogTo

El método LogTo permite configurar el destino de los mensajes de logging. Acepta un delegado que recibe el mensaje como cadena de texto. Se puede usar Console.WriteLine para escribir en la consola, o cualquier otro método que acepte una cadena.

```
optionsBuilder.LogTo(Console.WriteLine);
```

CSHARP

Esta línea escribe todos los mensajes en la consola. Para escribir en un archivo, se puede usar un StreamWriter.

```
var writer = new StreamWriter("efcore.log", append: true);
optionsBuilder.LogTo(writer.WriteLine);
```

CSHARP

La primera línea crea un escritor de archivo. La segunda configura el logging para que escriba en el archivo.

Niveles de logging

EF Core usa los niveles de logging estándar de .NET: Trace, Debug, Information, Warning, Error y Critical. Cada nivel indica la importancia del mensaje. El nivel Information incluye las sentencias SQL. El nivel Debug incluye información adicional sobre la ejecución. El nivel Warning incluye advertencias. El nivel Error incluye errores.

```
optionsBuilder.LogTo(Console.WriteLine, LogLevel.Information);
```

CSHARP

La primera línea configura el logging en la consola. La segunda especifica que solo se muestren los mensajes de nivel Information o superior. Los mensajes de nivel Debug y Trace no se muestran.

Filtrar mensajes por categoría

EF Core organiza los mensajes en categorías. Cada categoría corresponde a un componente interno. Las categorías más habituales son Microsoft.EntityFrameworkCore.Database.Command, Microsoft.EntityFrameworkCore.Query y Microsoft.EntityFrameworkCore.Update.

```
optionsBuilder.LogTo(
    Console.WriteLine,
    new[] { "Microsoft.EntityFrameworkCore.Database.Command" },
    LogLevel.Information);
```

CSHARP

La primera línea configura el logging en la consola. La segunda especifica las categorías que se quieren registrar. La tercera especifica el nivel mínimo. Solo se muestran los mensajes de la categoría Database.Command con nivel Information o superior.

Las categorías permiten filtrar los mensajes para centrarse en un aspecto concreto. La categoría Database.Command incluye las sentencias SQL. La categoría Query incluye información sobre la traducción de consultas. La categoría Update incluye información sobre las operaciones de escritura.

El método EnableSensitiveDataLogging

Por defecto, EF Core oculta los valores de los parámetros en los mensajes de logging para evitar exponer datos sensibles. El método EnableSensitiveDataLogging permite mostrar los valores reales de los parámetros.

```
optionsBuilder.EnableSensitiveDataLogging();
```

CSHARP

Esta línea habilita el logging de datos sensibles. Los parámetros de las consultas se muestran con sus valores reales. Es útil en desarrollo, pero no se recomienda en producción porque puede exponer datos confidenciales en los logs.

El método EnableDetailedErrors

Por defecto, EF Core no incluye información detallada en los mensajes de error para evitar exponer la estructura interna. El método EnableDetailedErrors permite mostrar información más detallada en los errores.

```
optionsBuilder.EnableDetailedErrors();
```

CSHARP

Esta línea habilita los errores detallados. Los mensajes de error incluyen información sobre las propiedades y las entidades implicadas. Es útil en desarrollo, pero puede exponer información interna en producción.

El método ConfigureWarnings

El método ConfigureWarnings permite configurar el comportamiento de EF Core ante determinadas advertencias. Se puede hacer que una advertencia se convierta en error, que se ignore o que se registre con un nivel distinto.

```
optionsBuilder.ConfigureWarnings(warnings =>
    warnings.Throw(RelationalEventId.MultipleCollectionIncludeWarning));
```

CSHARP

La primera línea configura las advertencias. La segunda hace que la advertencia MultipleCollectionIncludeWarning se convierta en excepción. Esta advertencia se produce cuando se incluyen varias colecciones en una misma consulta y puede provocar un producto cartesiano.

El método UseQueryTrackingBehavior

El método UseQueryTrackingBehavior permite configurar el comportamiento por defecto del seguimiento de cambios en las consultas. Se puede establecer en TrackAll (por defecto) o en NoTracking.

```
optionsBuilder.UseQueryTrackingBehavior(QueryTrackingBehavior.NoTracking);
```

CSHARP

Esta línea establece que todas las consultas se ejecuten sin seguimiento de cambios por defecto. Es útil en aplicaciones de solo lectura donde no se van a modificar los datos.

El método UseLazyLoadingProxies

El método UseLazyLoadingProxies habilita la carga diferida de propiedades de navegación. Requiere el paquete Microsoft.EntityFrameworkCore.Proxies.

```
optionsBuilder.UseLazyLoadingProxies();
```

CSHARP

Esta línea habilita la carga diferida. Las propiedades de navegación se cargan automáticamente cuando se accede a ellas. Este comportamiento se estudiará en detalle en módulos posteriores.

El método UseSqlServer con opciones

El método UseSqlServer acepta un delegado que permite configurar opciones específicas del proveedor de SQL Server, como el tiempo de espera de comandos, el número de reintentos y el comportamiento ante errores de conexión.

```
optionsBuilder.UseSqlServer(
    "Server=(localdb)\\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;",
    sqlOptions =>
    {
        sqlOptions.EnableRetryOnFailure(maxRetryCount: 5);
        sqlOptions.CommandTimeout(60);
    });
```

CSHARP

La primera línea configura el proveedor de SQL Server. La segunda abre el delegado de configuración. La tercera habilita los reintentos automáticos en caso de error transitorio, con un máximo de cinco intentos. La cuarta establece el tiempo de espera de comandos en sesenta segundos.

El proyecto AceriaData

En el proyecto AceriaData, se añade un archivo appsettings.json con la cadena de conexión, se configura el DbContext para leerla, se habilita el logging con filtros por categoría y se configuran las opciones del proveedor de SQL Server. La base de datos sigue siendo AceriaDB en SQL Server LocalDB.

Resumen de la teoría

DbContextOptions agrupa todas las opciones de configuración del DbContext.

Las opciones se pueden pasar por constructor o configurar en OnConfiguring.

La condición IsConfigured evita sobrescribir la configuración existente.

La cadena de conexión contiene los datos de conexión al servidor y a la base de datos.

La cadena se puede almacenar en appsettings.json o en variables de entorno.

El logging registra las operaciones de EF Core.

El método LogTo configura el destino del logging.

Los niveles de logging indican la importancia del mensaje.

Las categorías permiten filtrar los mensajes por componente.

EnableSensitiveDataLogging muestra los valores de los parámetros.

EnableDetailedErrors muestra información detallada en los errores.

ConfigureWarnings configura el comportamiento ante advertencias.

UseQueryTrackingBehavior configura el seguimiento por defecto.

UseSqlServer acepta un delegado para configurar opciones del proveedor.

Punto 1.11 - Proveedores de datos: SQLite, SQL Server y PostgreSQL

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core.

Proyecto: Se comparan conceptualmente los proveedores de datos de EF Core, mientras AceriaData permanece configurado exclusivamente con SQL Server LocalDB.

Objetivos de aprendizaje

Comprender qué es un proveedor de datos en EF Core y cuál es su responsabilidad.

Conocer los proveedores oficiales más habituales: SQL Server, SQLite y PostgreSQL.

Identificar las diferencias entre motores y su impacto en el modelo.

Configurar el proveedor de SQL Server en el proyecto AceriaData.

Reconocer las diferencias conceptuales de SQLite y PostgreSQL sin cambiar el proveedor del proyecto.

Distinguir los proveedores operativos de los proveedores utilizados como dobles de prueba y comprender sus limitaciones.

Aplicar la configuración del proveedor de SQL Server al proyecto AceriaData.

Teoría

Qué es un proveedor de datos

Un proveedor de datos es el componente de EF Core que traduce las operaciones abstractas del ORM al lenguaje específico de un motor de base de datos concreto. EF Core no sabe hablar SQL Server, ni SQLite, ni PostgreSQL. Sabe construir un árbol de expresión que describe lo que se quiere hacer. El proveedor es el encargado de convertir ese árbol en sentencias SQL válidas para el motor correspondiente, ejecutarlas y materializar los resultados.

Cada proveedor se distribuye como un paquete NuGet independiente. El paquete Microsoft.EntityFrameworkCore.SqlServer contiene el proveedor para SQL Server y Azure SQL. El paquete Microsoft.EntityFrameworkCore.Sqlite contiene el proveedor para SQLite. El paquete Npgsql.EntityFrameworkCore.PostgreSQL contiene el proveedor para PostgreSQL. El paquete Pomelo.EntityFrameworkCore.MySql contiene el proveedor para MySQL y MariaDB.

```
optionsBuilder.UseSqlServer("Server=(localdb)\\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;");
```

CSHARP

La primera línea configura el proveedor de SQL Server. La cadena de conexión indica la instancia y la base de datos. Otros proveedores usan métodos de extensión propios. En AceriaData, los ejemplos ejecutables de este módulo continúan sobre SQL Server.

Por qué existen varios proveedores

Cada motor de base de datos tiene sus propias características. SQL Server es un motor empresarial con soporte para transacciones distribuidas, procedimientos almacenados y tipos de datos avanzados. SQLite es un motor embebido, sin servidor, ideal para aplicaciones de escritorio, móviles y pruebas. PostgreSQL es un motor open source con soporte para tipos avanzados como JSONB, arrays y rangos. MySQL es un motor muy extendido en aplicaciones web.

Un proveedor debe conocer las particularidades del motor para generar SQL correcto. La sintaxis de paginación es distinta: SQL Server usa OFFSET ... FETCH, SQLite usa LIMIT ... OFFSET, PostgreSQL usa LIMIT ... OFFSET. La sintaxis para generar claves es distinta: SQL Server usa IDENTITY, SQLite usa AUTOINCREMENT, PostgreSQL usa SERIAL o GENERATED. El proveedor encapsula muchas de estas diferencias, pero no garantiza portabilidad total: tipos, funciones, semántica de consultas y migraciones pueden exigir cambios.

El proveedor de SQL Server

El proveedor de SQL Server se distribuye en el paquete Microsoft.EntityFrameworkCore.SqlServer. Internamente utiliza Microsoft.Data.SqlClient para abrir conexiones y ejecutar comandos. Es compatible con SQL Server 2012 y versiones posteriores, con Azure SQL Database y con SQL Server Express LocalDB.

La cadena de conexión tiene el siguiente formato:

```
Server=(localdb)\\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;
```

TEXT

Server → indica la instancia de SQL Server. (localdb)\\mssqllocaldb es la instancia por defecto de LocalDB.

Database → nombre de la base de datos. Si no existe, EF Core la crea cuando se llama a EnsureCreated o se aplican migraciones.

Trusted_Connection=True → usa la autenticación de Windows del usuario actual.

SQL Server soporta tipos de datos como int, bigint, decimal, nvarchar, datetime2, bit y uniqueidentifier. EF Core mapea los tipos de C# a estos tipos de SQL Server. Por ejemplo, string se mapea a nvarchar(max) por defecto, int a int, DateTime a datetime2 y bool a bit.

```
public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
    public decimal PesoTotal { get; set; }
    public DateTime FechaCreacion { get; set; }
    public bool Activa { get; set; }
}
```

CSHARP

En SQL Server, esta entidad se traduce a una tabla con las columnas Id de tipo int, NumeroOrden de tipo nvarchar(max), PesoTotal de tipo decimal(18,2), FechaCreacion de tipo datetime2 y Activa de tipo bit.

El proveedor de SQLite

El proveedor de SQLite se distribuye en el paquete `Microsoft.EntityFrameworkCore.Sqlite`. Internamente utiliza `Microsoft.Data.Sqlite` para abrir conexiones y ejecutar comandos. SQLite es un motor embebido que almacena la base de datos en un archivo local. No requiere instalación de servidor ni configuración de red.

La cadena de conexión tiene el siguiente formato:

En SQLite, la cadena de conexión suele identificar un archivo local mediante `Data Source`. `AceriaData` no configura una conexión SQLite en este módulo.

SQLite tiene un sistema de tipos más flexible que SQL Server. Los tipos son INTEGER, REAL, TEXT, BLOB y NUMERIC. EF Core mapea los tipos de C# a estos tipos. `string` se mapea a TEXT, `int` a INTEGER, `double` a REAL, `DateTime` a TEXT y `bool` a INTEGER.

```
public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
    public decimal PesoTotal { get; set; }
    public DateTime FechaCreacion { get; set; }
    public bool Activa { get; set; }
}
```

CSHARP

En SQLite, esta entidad se traduce a una tabla con las columnas `Id` de tipo INTEGER, `NumeroOrden` de tipo TEXT, `PesoTotal` de tipo TEXT, `FechaCreacion` de tipo TEXT y `Activa` de tipo INTEGER. La diferencia más notable es que decimal se almacena como TEXT, lo que puede provocar problemas de precisión si no se configura explícitamente.

El proveedor de PostgreSQL

El proveedor de PostgreSQL se distribuye en el paquete `Npgsql.EntityFrameworkCore.PostgreSQL`, mantenido por la comunidad. Internamente utiliza `Npgsql` para abrir conexiones y ejecutar comandos. PostgreSQL es un motor open source muy usado en aplicaciones empresariales por su robustez y sus características avanzadas.

La cadena de conexión tiene el siguiente formato:

PostgreSQL utiliza una cadena de conexión con host, base de datos y credenciales. Se describe aquí sólo para comprender el papel de los proveedores; `AceriaData` seguirá usando SQL Server.

PostgreSQL soporta tipos avanzados como `jsonb`, `uuid`, `array`, `range` y `inet`. EF Core mapea los tipos de C# a estos tipos. `string` se mapea a text, `int` a integer, `DateTime` a timestamp with time zone y `bool` a boolean. Los tipos avanzados requieren configuración explícita.

Diferencias entre motores

Las diferencias entre motores afectan al modelo y a las consultas. La primera diferencia es el sistema de tipos. SQL Server usa `nvarchar`, SQLite usa TEXT y PostgreSQL usa text. La segunda diferencia es la generación de claves. SQL Server usa IDENTITY, SQLite usa AUTOINCREMENT y PostgreSQL usa SERIAL. La tercera diferencia es la sintaxis de paginación. La cuarta diferencia es el soporte de transacciones. La quinta diferencia es el comportamiento de las funciones de fecha y hora.

```
var ordenes = context.OrdenesFabricacion
    .OrderBy(o => o.FechaCreacion)
    .Skip(10)
    .Take(5)
    .ToList();
```

CSHARP

Esta consulta se traduce de forma distinta según el proveedor. En SQL Server, se genera `OFFSET 10 ROWS FETCH NEXT 5 ROWS ONLY`. En SQLite, se genera `LIMIT 5 OFFSET 10`. En PostgreSQL, se genera `LIMIT 5 OFFSET 10`. El código de la aplicación es el mismo, pero el SQL generado es distinto.

Configurar el proveedor

El proveedor se configura al construir el `DbContext`. Se llama al método de extensión correspondiente sobre `DbContextOptionsBuilder`. El método `UseSqlServer` registra el proveedor de SQL Server. El método `UseSqlite` registra el proveedor de SQLite. El método `UseNpgsql` registra el proveedor de PostgreSQL.

```
optionsBuilder.UseSqlServer("Server=(localdb)\\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;");
```

La primera línea registra el proveedor de SQL Server con la cadena de conexión. A partir de este momento, todas las operaciones se traducen a SQL Server.

Cambiar de proveedor

Cambiar de proveedor requiere cambiar la llamada al método de extensión y la cadena de conexión. El resto del código de la aplicación no cambia si el modelo no usa características específicas de un motor. Sin embargo, las migraciones sí son específicas del proveedor. Si se cambia de SQL Server a SQLite, las migraciones generadas para SQL Server no son válidas para SQLite y deben regenerarse.

El cambio de proveedor requeriría usar el método de extensión correspondiente y revisar las migraciones, pero no se realizará en AceriaData.

El proveedor en memoria

EF Core dispone de un proveedor InMemory pensado como doble de prueba, no como sustituto relacional de SQL Server. No ejecuta SQL ni reproduce muchas restricciones y comportamientos relacionales. Su uso, limitaciones y comparación con SQLite in-memory pertenecen al bloque específico de testing.

SQLite en modo en memoria

SQLite puede utilizarse en memoria como doble relacional de prueba, pero su semántica y traducción siguen siendo distintas de SQL Server. No se configura como proveedor operativo de AceriaData en este módulo.

El proveedor en el proyecto AceriaData

En el proyecto AceriaData, el único proveedor configurado en este punto es SQL Server LocalDB. La comparación con otros motores es conceptual y no altera ni el código ejecutable ni las migraciones del proyecto.

Resumen de la teoría

Un proveedor de datos traduce las operaciones de EF Core a SQL específico del motor.

Cada proveedor se distribuye como un paquete NuGet independiente.

Los proveedores oficiales son SQL Server, SQLite y PostgreSQL.

Cada motor tiene su propio sistema de tipos y su propia sintaxis.

Cambiar de proveedor requiere cambiar la llamada al método de extensión y la cadena de conexión.

Las migraciones son específicas del proveedor.

InMemory y SQLite in-memory se estudiarán en el bloque específico de testing, donde se analizarán como dobles de prueba y se explicarán sus limitaciones.

En el proyecto AceriaData se mantiene SQL Server LocalDB como único proveedor operativo durante este módulo.

Punto 1.12 - Integración de EF Core en aplicaciones .NET: inyección de dependencias y AddDbContext

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core.

Proyecto: Se refactoriza el proyecto AceriaData para usar inyección de dependencias con Microsoft.Extensions.DependencyInjection y AddDbContext, manteniendo SQL Server LocalDB como proveedor.

Objetivos de aprendizaje

Comprender qué es la inyección de dependencias y por qué se usa.

Conocer el contenedor de servicios de .NET.

Registrar el DbContext con AddDbContext y AddDbContextFactory.

Diferenciar los ciclos de vida Singleton, Scoped y Transient.

Inyectar el DbContext en servicios y repositorios.

Configurar el contenedor en una aplicación de consola.

Aplicar estos conceptos al proyecto AceriaData.

Teoría

Qué es la inyección de dependencias

La inyección de dependencias es un patrón de diseño que consiste en proporcionar a un objeto las dependencias que necesita desde fuera, en lugar de que el objeto las cree por sí mismo. En lugar de que una clase instancie sus dependencias con new, las recibe a través de su constructor, de sus propiedades o de sus métodos. Este patrón desacopla las clases, facilita el testing y permite sustituir implementaciones sin modificar el código que las usa.

```
public class ServicioOrdenes
{
    private readonly AceriaDbContext _context;

    public ServicioOrdenes(AceriaDbContext context)
    {
        _context = context;
    }

    public List<OrdenFabricacion> ObtenerTodas()
    {
        return _context.OrdenesFabricacion.ToList();
    }
}
```

CSHARP

La primera línea declara el campo _context. La segunda declara el constructor que recibe el DbContext. La tercera asigna el parámetro al campo. La cuarta declara el método que usa el DbContext. El servicio no crea el DbContext: lo recibe desde fuera.

Por qué usar inyección de dependencias

Sin inyección de dependencias, cada clase crea sus propias dependencias. Esto provoca acoplamiento fuerte entre las clases, dificulta el testing y hace que los cambios en una dependencia afecten a todas las clases que la usan.

```
public class ServicioOrdenesSinDI
{
    public List<OrdenFabricacion> ObtenerTodas()
    {
        using var context = new AceriaDbContext("...", "SqlServer");
        return context.OrdenesFabricacion.ToList();
    }
}
```

CSHARP

La primera línea declara el método. La segunda crea el DbContext directamente. La tercera consulta las órdenes. Esta clase está acoplada al DbContext y a su cadena de conexión. Si se quiere cambiar la cadena o usar otro contexto, hay que modificar la clase.

Con inyección de dependencias, el servicio recibe el DbContext desde fuera y no sabe cómo se crea. Esto permite cambiar la implementación sin modificar el servicio.

El contenedor de servicios de .NET

El contenedor de servicios es el componente que gestiona la creación y el ciclo de vida de los objetos registrados. En .NET, el contenedor se construye con ServiceCollection y se materializa con BuildServiceProvider. Una vez construido, se pueden resolver los servicios registrados.

```
var services = new ServiceCollection();
services.AddDbContext<AceriaDbContext>(options =>
    options.UseSqlServer("Server=(localdb)\\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;"));
var provider = services.BuildServiceProvider();

using var scope = provider.CreateScope();
var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
```

CSHARP

La primera línea crea la colección de servicios. La segunda registra el DbContext con AddDbContext. La tercera construye el proveedor de servicios. La cuarta crea un ámbito. La quinta resuelve el DbContext desde el ámbito.

El método AddDbContext

El método AddDbContext registra el DbContext en el contenedor con ciclo de vida Scoped. Esto significa que se crea una instancia nueva por cada ámbito. En aplicaciones web, cada petición HTTP crea un ámbito. En aplicaciones de consola, cada llamada a CreateScope crea un ámbito nuevo.

```
services.AddDbContext<AceriaDbContext>(options =>
    options.UseSqlServer("Server=(localdb)\\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;"));
```

CSHARP

La primera línea registra el DbContext. La segunda configura el proveedor de SQL Server con la cadena de conexión. El DbContext se crea cada vez que se resuelve desde un ámbito.

El método AddDbContextFactory

El método AddDbContextFactory registra una fábrica de DbContext en el contenedor con ciclo de vida Singleton. La fábrica se puede inyectar en servicios de larga duración y crear contextos de corta duración bajo demanda.

```
services.AddDbContextFactory<AceriaDbContext>(options =>
    options.UseSqlServer("Server=(localdb)\\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;"));
```

CSHARP

La primera línea registra la fábrica. La segunda configura el proveedor de SQL Server. La fábrica se inyecta como IDbContextFactory<AceriaDbContext> y se llama a CreateDbContext para obtener una instancia nueva.

Ciclos de vida de los servicios

El contenedor de servicios de .NET soporta tres ciclos de vida. El ciclo Singleton crea una única instancia durante toda la vida de la aplicación. El ciclo Scoped crea una instancia por ámbito. El ciclo Transient crea una instancia nueva cada vez que se resuelve el servicio.

```
services.AddSingleton<IServicioSingleton, ServicioSingleton>();
services.AddScoped<IServicioScoped, ServicioScoped>();
services.AddTransient<IServicioTransient, ServicioTransient>();
```

CSHARP

La primera línea registra un servicio singleton. La segunda registra un servicio con ámbito. La tercera registra un servicio transitorio. La elección del ciclo de vida depende de la naturaleza del servicio y de sus dependencias.

El DbContext se registra con Scoped por defecto porque no es seguro para subprocessos y debe ser de corta duración. Un DbContext Singleton acumularía entidades en el Change Tracker y devolvería datos obsoletos.

El método BuildServiceProvider

El método BuildServiceProvider construye el proveedor de servicios a partir de la colección de servicios. Una vez construido, el proveedor se usa para resolver servicios y crear ámbitos.

```
var provider = services.BuildServiceProvider();
```

CSHARP

La primera línea construye el proveedor. A partir de este momento, se pueden resolver servicios con GetRequiredService o GetService.

El método CreateScope

El método CreateScope crea un ámbito nuevo. El ámbito agrupa los servicios con ciclo de vida Scoped. Al liberar el ámbito, se liberan todos los servicios creados dentro de él.

```
using var scope = provider.CreateScope();
var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
```

CSHARP

La primera línea crea un ámbito. La segunda resuelve el DbContext desde el ámbito. Al salir del bloque using, el ámbito se libera y con él el DbContext.

El método GetRequiredService

El método `GetRequiredService<T>` resuelve un servicio del contenedor. Si el servicio no está registrado, lanza una excepción `InvalidOperationException`. El método `GetService<T>` devuelve null si el servicio no está registrado.

```
var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
```

CSHARP

La primera línea resuelve el `DbContext`. Si no está registrado, lanza una excepción.

Injectar servicios en otros servicios

Los servicios se pueden inyectar en otros servicios a través del constructor. El contenedor resuelve las dependencias de forma recursiva.

```
public class ServicioOrdenes
{
    private readonly AceriaDbContext _context;

    public ServicioOrdenes(AceriaDbContext context)
    {
        _context = context;
    }
}

services.AddScoped<ServicioOrdenes>();
```

CSHARP

La primera línea declara la clase. La segunda declara el campo. La tercera declara el constructor. La cuarta asigna el parámetro. La quinta registra el servicio en el contenedor. El contenedor resuelve el `DbContext` y lo pasa al constructor.

El patrón repositorio con inyección de dependencias

El patrón repositorio encapsula el acceso a datos en una clase que expone métodos de consulta y escritura. El repositorio recibe el `DbContext` por constructor y lo usa internamente.

```
public interface IOordenRepositorio
{
    List<OrdenFabricacion> ObtenerTodas();
    OrdenFabricacion? ObtenerPorId(int id);
    void Agregar(OrdenFabricacion orden);
    void Guardar();
}

public class OrdenRepositorio : IOordenRepositorio
{
    private readonly AceriaDbContext _context;

    public OrdenRepositorio(AceriaDbContext context)
    {
        _context = context;
    }

    public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.ToList();

    public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);

    public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);

    public void Guardar() => _context.SaveChanges();
}
```

CSHARP

La primera línea declara la interfaz. La segunda declara el método que devuelve todas las órdenes. La tercera declara el método que devuelve una orden por Id. La cuarta declara el método que agrega una orden. La quinta declara el método que guarda los cambios. La sexta declara la implementación. La séptima declara el campo. La octava declara el constructor. La novena asigna el parámetro. Las siguientes líneas implementan los métodos.

Registro del repositorio en el contenedor

El repositorio se registra en el contenedor con su interfaz. El contenedor resuelve el `DbContext` y lo pasa al constructor del repositorio.

```
services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();
```

CSHARP

La primera línea registra la interfaz y su implementación con ciclo de vida Scoped. El contenedor crea una instancia nueva por ámbito.

Inyección en una aplicación de consola

En una aplicación de consola, se construye la colección de servicios, se registran las dependencias y se construye el proveedor. Después, se crea un ámbito por cada operación y se resuelven los servicios desde el ámbito.

```
var services = new ServiceCollection();
services.AddDbContext<AceriaDbContext>(options =>
    options.UseSqlServer("Server=(localdb)\\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;"));
services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();

var provider = services.BuildServiceProvider();

using var scope = provider.CreateScope();
var repositorio = scope.ServiceProvider.GetRequiredService<IOrdenRepositorio>();

var ordenes = repositorio.ObtenerTodas();
```

CSHARP

La primera línea crea la colección. La segunda registra el DbContext. La tercera registra el repositorio. La cuarta construye el proveedor. La quinta crea un ámbito. La sexta resuelve el repositorio. La séptima consulta las órdenes.

Inyección en ASP.NET Core

En ASP.NET Core, el contenedor de servicios se construye automáticamente con WebApplicationBuilder. Los servicios se registran en la propiedad Services del builder y se inyectan en los endpoints o en los controladores.

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddDbContext<AceriaDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("AceriaDB")));
builder.Services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();

var app = builder.Build();
```

CSHARP

La primera línea crea el builder. La segunda registra el DbContext con la cadena de conexión leída de la configuración. La tercera registra el repositorio. La cuarta construye la aplicación.

El método AddDbContextPool

El método AddDbContextPool registra el DbContext en un pool de instancias reutilizables. Reduce el coste de crear y liberar contextos en aplicaciones de alta concurrencia.

```
services.AddDbContextPool<AceriaDbContext>(options =>
    options.UseSqlServer("Server=(localdb)\\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;"));
```

CSHARP

La primera línea registra el DbContext en un pool. Las instancias se reutilizan entre ámbitos. Es útil en aplicaciones web con muchas peticiones por segundo.

El proyecto AceriaData

En el proyecto AceriaData, se refactoriza el código para usar inyección de dependencias con Microsoft.Extensions.DependencyInjection. Se registra el DbContext con AddDbContext, se crea un repositorio para las órdenes y se resuelven los servicios desde el contenedor. La base de datos sigue siendo AceriaDB en SQL Server LocalDB.

Resumen de la teoría

La inyección de dependencias proporciona las dependencias desde fuera.

El contenedor de servicios gestiona la creación y el ciclo de vida de los objetos.

AddDbContext registra el DbContext con ciclo de vida Scoped.

AddDbContextFactory registra una fábrica de DbContext con ciclo de vida Singleton.

Los ciclos de vida son Singleton, Scoped y Transient.

CreateScope crea un ámbito que agrupa los servicios Scoped.

GetRequiredService resuelve un servicio del contenedor.

Los servicios se inyectan en otros servicios por constructor.

El patrón repositorio encapsula el acceso a datos.

En aplicaciones de consola se construye el contenedor manualmente.

En ASP.NET Core el contenedor se construye con WebApplicationBuilder.

AddDbContextPool reutiliza instancias del DbContext.